

ESTRUCTURAS DE DATOS Y ALGORITMOS

ALFRED V. VHO
JOHN E. HEDRUMST
JERRY D. SALAS



Estructura de datos y algoritmos

Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman
con la colaboración de Guillermo Levine Gutiérrez;
versión en español de Américo Vargas y Jorge Lozano
PUBLICACIÓN México, DF : Addison-Wesley
Iberoamericana: Sistemas Técnicos de Edición, 1988
ISBN 968-6048-19-7

1

Diseño y análisis de algoritmos

Escribir un programa de computador para resolver un problema comprende varios pasos que van desde la formulación y especificación del problema, el diseño de la solución, su implantación, prueba y documentación, hasta la evaluación de la solución. En este capítulo se presenta el punto de vista de los autores sobre estos pasos. En los capítulos siguientes se analizan los algoritmos y estructuras de datos con los que se construye la mayor parte de los programas de computador.

1.1 De los problemas a los programas

La mitad del trabajo es saber qué problema se va a resolver. Al abordar los problemas, por lo general, éstos no tienen una especificación simple y precisa de ellos. De hecho, problemas como crear una receta digna de un gastrónomo o preservar la paz mundial pueden ser imposibles de formular de forma que admitan una solución por computador; aunque se crea que el problema puede resolverse en un computador, es usual que la distancia entre varios de sus parámetros sea considerable. A menudo sólo mediante experimentación es posible encontrar valores razonables para estos parámetros.

Si es posible expresar ciertos aspectos de un problema con un modelo formal, por lo general resulta beneficioso hacerlo, pues una vez que el problema se formaliza, pueden buscarse soluciones en función de un modelo preciso y determinar si ya existe un programa que resuelva tal problema; aun cuando no sea tal el caso, será posible averiguar lo que se sabe acerca del modelo y usar sus propiedades como ayuda para elaborar una buena solución.

Se puede recurrir casi a cualquier rama de las matemáticas y de las ciencias para obtener un modelo de cierto tipo de problemas. En el caso de problemas de naturaleza esencialmente numérica, esto puede lograrse a través de conceptos matemáticos tan familiares como las ecuaciones lineales simultáneas (por ejemplo, para determinar las corrientes en un circuito eléctrico o encontrar los puntos de tensión en estructuras de vigas conectadas) o ecuaciones diferenciales (por ejemplo, para predecir el crecimiento de una población o la velocidad de una reacción química). Tratándose de problemas de procesamiento de símbolos y textos, se pueden construir modelos con cadenas de caracteres y gramáticas formales. Entre los problemas de esta categoría están la compilación (traducción a lenguaje de máquina de programas escritos en algún lenguaje de programación) y las tareas de recuperación de infor-

mación, como el reconocimiento de ciertas palabras en el catálogo de títulos de una biblioteca.

Algoritmos

Cuando ya se cuenta con un modelo matemático adecuado del problema, puede buscarse una solución en función de ese modelo. El objetivo inicial consiste en hallar una solución en forma de *algoritmo*, que es una secuencia finita de instrucciones, cada una de las cuales tiene un significado preciso y puede ejecutarse con una cantidad finita de esfuerzo en un tiempo finito. Una proposición de asignación entera como $x := y + z$ es un ejemplo de instrucción que puede ejecutarse con una cantidad finita de esfuerzo. Las instrucciones de un algoritmo pueden ejecutarse cualquier número de veces, siempre que ellas mismas indiquen la repetición. No obstante, se exige que un algoritmo termine después de ejecutar un número finito de instrucciones, sin importar cuáles fueron los valores de entrada. Así, un programa es un algoritmo mientras no entre en un ciclo infinito con ninguna entrada.

Es necesario aclarar un aspecto de esta definición de algoritmo. Se ha dicho que cada instrucción de un algoritmo debe tener un «significado preciso» y debe ser ejecutable con una «cantidad finita de esfuerzo»; pero lo que está claro para una persona, puede no estarlo para otra, y a menudo es difícil demostrar de manera rigurosa que una instrucción puede realizarse en un tiempo finito. A veces, también es difícil demostrar que una secuencia de instrucciones termina con alguna entrada, aunque esté muy claro el significado de cada instrucción. Sin embargo, al argumentar en favor y en contra, por lo general se llega a un acuerdo respecto a si una secuencia de instrucciones constituye o no un algoritmo. Quien afirme tener un algoritmo debe asumir la responsabilidad de demostrarlo. En la sección 1.5 se verá cómo estimar el tiempo de ejecución de algunas construcciones usuales con lenguajes de programación de las que se puede demostrar que requieren de un tiempo finito de ejecución.

Además de usar programas en Pascal como algoritmos, se empleará un *seudolenguaje* que es una combinación de las construcciones de un lenguaje formal de programación con proposiciones informales expresadas en español. Se utilizará Pascal como lenguaje de programación, pero para realizar los algoritmos que se estudiarán, éste puede sustituirse casi por cualquier otro lenguaje conocido. El ejemplo que se da a continuación ilustra muchos de los pasos que componen la escritura de un programa de computador, de acuerdo con el esquema de los autores.

Ejemplo 1.1. Un modelo matemático puede ayudar a diseñar un semáforo para una intersección complicada de calles. Para construir la secuencia de señales, se creará un programa que acepte como entrada el conjunto de giros permisibles en una intersección (un «giro» también será continuar en línea recta por una calle) y que divida este conjunto en el menor número posible de grupos, de manera que todos los giros de un grupo sean permisibles en forma simultánea sin colisiones. Después, se asociará una señal del semáforo a cada grupo de giros. Buscando una división que tenga el menor número posible de grupos, se obtendrá un semáforo con el menor número posible de fases.

Se tomará como ejemplo la intersección de la figura 1.1, donde las calles *C* y *E*

son de una dirección, mientras las restantes son de dos. En esta intersección es posible realizar 13 giros. Algunos pares de giros, como AB (de A hacia B) y EC , pueden ocurrir en forma simultánea, mientras que otros, como AD y EB , ocasionan el cruce entre líneas de tráfico y, por tanto, no pueden realizarse al mismo tiempo. Así, las luces del semáforo deben permitir los giros en un orden tal que AD y EB nunca ocurran al mismo tiempo, en tanto que AB y EC puedan ser simultáneos.

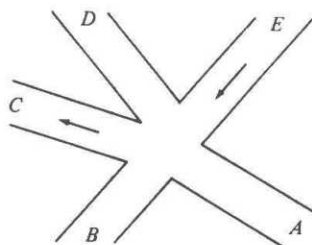


Fig. 1.1. Una intersección.

Puede obtenerse un modelo de este problema con la ayuda de una estructura matemática conocida como *grafo*. Un grafo se compone de un conjunto de puntos, llamados *vértices*, y de líneas que unen los puntos, llamadas *aristas*. Es posible construir un modelo del problema de la intersección de tráfico mediante un grafo cuyos vértices representen los giros posibles y cuyos arcos unan los pares de vértices que representan giros que no se permiten al mismo tiempo. En la figura 1.2 se muestra el grafo para la intersección de la figura 1.1. La figura 1.3 es otra representación del mismo grafo en forma de tabla, donde un 1 en la intersección de la fila i y la columna j indica que hay una arista entre los vértices i y j .

El grafo puede ayudar a solucionar el problema de diseñar el semáforo. La *coloración* de un grafo es la asignación de un color a cada vértice de éste, de tal forma que no haya dos vértices del mismo color unidos por un arco. Así pues, resulta evidente que el problema equivale a buscar una coloración del grafo de giros incompatibles que utilice el menor número posible de colores.

El problema de la coloración de grafos se ha estudiado durante varias décadas, la teoría de algoritmos ofrece mucha información. Lamentablemente, la coloración de un grafo arbitrario con el menor número posible de colores es un problema que se clasifica entre los llamados «NP-completos», para los cuales las únicas soluciones conocidas consisten, en esencia, en «intentar todas las posibilidades». En el problema de la coloración, esto significa intentar todas las asignaciones de colores a los vértices, primero con un solo color, luego con dos, con tres, y así sucesivamente, hasta encontrar una coloración válida. Con un poco de cuidado es posible acelerar este procedimiento, pero, en general, se cree que ningún algoritmo que resuelva este problema puede ser sustancialmente más eficiente que el enfoque, más obvio, recién descrito.

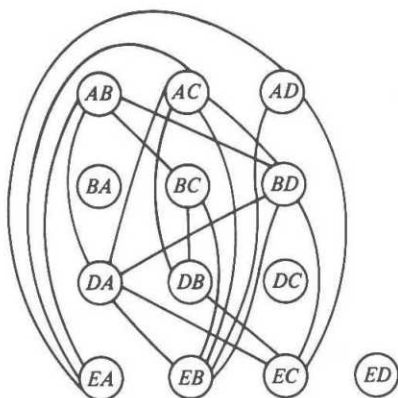


Fig. 1.2. Grafo que representa los giros incompatibles.

	AB	AC	AD	BA	BC	BD	DA	DB	DC	EA	EB	EC	ED
AB					1	1	1	1		1			
AC						1	1	1		1	1		
AD										1	1	1	
BA													
BC	1							1			1		
BD	1	1					1				1	1	
DA	1	1				1					1	1	
DB		1			1							1	
DC													
EA	1	1	1										
EB		1	1		1	1	1						
EC			1			1	1	1					
ED													

Fig. 1.3. Tabla de giros incompatibles.

Ahora, existe la posibilidad de que encontrar una solución óptima al problema en cuestión tenga un costo computacional muy elevado. Ante esto, puede elegirse entre tres enfoques. Si el grafo es pequeño se puede buscar una solución óptima a través de una búsqueda exhaustiva, intentando todas las posibilidades. Para grafos grandes, en cambio, este enfoque resulta prohibitivo por su costo, sin importar qué gra-

do de eficiencia se intente dar al programa. El segundo enfoque consiste en buscar información adicional sobre el problema. Podría suceder que el grafo tuviera propiedades especiales que hicieran innecesario probar todas las posibilidades para hallar una solución óptima. El tercer enfoque consiste en cambiar un poco el problema y buscar una solución buena aunque no necesariamente óptima; podría ser suficiente con encontrar una solución que determinara un número de colores cercano al mínimo en grafos pequeños, y que trabajara con rapidez, pues la mayor parte de las intersecciones no son tan complejas como las de la figura 1.1. Un algoritmo que produce con rapidez soluciones buenas, pero no necesariamente óptimas, se denomina *heurístico*.

El siguiente algoritmo «ávido» es un heurístico razonable para la coloración de grafos. Para empezar, se intenta colorear tantos vértices como sea posible con el primer color; después, todos los que sea posible con el segundo color, y así sucesivamente. Para dar a los vértices un nuevo color, se procede como sigue:

1. Se selecciona un vértice no coloreado y se le asigna el nuevo color.
2. Se examina la lista de vértices no coloreados. Para cada uno de ellos se determina si existe alguna arista que lo una con un vértice que ya tenga asignado el nuevo color; si tal arista no existe, se asigna el nuevo color al vértice.

Este enfoque recibe el nombre «ávido» porque le asigna color a un vértice en cuanto le es posible, sin considerar las potenciales desventajas inherentes a tal acción. Hay situaciones en las que podrían colorearse más vértices con un mismo color si fuera menos «ávido» y pasara por alto algún vértice que legalmente se podría colorear. Por ejemplo, considérese el grafo de la figura 1.4, donde, después de colorear con rojo el vértice 1, se puede asignar el mismo color a los vértices 3 y 4, siempre que no se coloree antes el vértice 2. Si se consideran los vértices en orden numérico, con el algoritmo exhaustivo se asignaría el color rojo a los vértices 1 y 2.

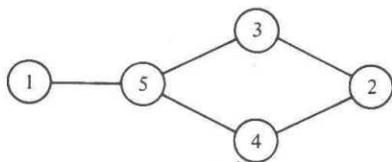


Fig. 1.4. Un grafo.

Como ejemplo del enfoque ávido aplicado a la figura 1.2, supóngase que se empieza por asignar a AB el color azul. Se pueden colorear AC , AD y BA de azul, pues ninguno de estos tres vértices tiene una arista común con AB . No es posible asignar el color azul al vértice BC , porque existe una arista entre AB y BC . Del mismo modo, no se pueden colorear de azul BD , DA ni DB , porque están unidos por una arista a uno o más vértices de los ya coloreados. Sin embargo, sí se puede asignar el azul a DC . Así, EA , EB y EC no se pueden colorear con azul, pero ED sí.

Ahora se empieza con el segundo color, por ejemplo, coloreando BC de rojo. BD puede colorearse de rojo, pero DA no, a causa de la arista entre BD y DA . De igual manera, a DB no se le puede asignar el color rojo y DC ya es azul, pero EA sí se puede colorear de rojo. Los demás vértices sin color tienen ahora una arista con un vértice rojo, de modo que ningún otro vértice puede ser rojo.

Los vértices que aún no tienen color son DA , DB , EB y EC . Si se colorea DA de verde, DB también podrá ser verde, pero EB y EC no. Estos deben colorearse con un cuarto color, por ejemplo, amarillo. La asignación de colores se resume en la figura 1.5. Al aplicar el enfoque ávido se determina que los giros «adicionales» de cierto color son compatibles con aquellos que ya tenían asignado ese mismo color y entre sí. Cuando el semáforo permita giros de un color, también permitirá los giros adicionales sin colisiones.

color	giros	giros adicionales
azul	AB, AC, AD, BA, DC, ED	—
rojo	BC, BD, EA	BA, DC, ED
verde	DA, DB	AD, BA, DC, ED
amarillo	EB, EC	BA, DC, EA, ED

Fig. 1.5. Coloración del grafo de la figura 1.2.

El enfoque ávido no siempre usa el menor número posible de colores. Ahora, es posible utilizar de nuevo la teoría de algoritmos para evaluar la bondad de la solución obtenida. En teoría de grafos, un *grafo completo- k* es un conjunto de k vértices donde cada par de ellos está unido por una arista. Puesto que en un grafo completo no hay dos vértices a los que se pueda asignar el mismo color, es obvio que se necesitan k colores para colorear un grafo completo- k .

En el grafo de la figura 1.2, el conjunto de cuatro vértices AC , DA , BD y EB es un grafo completo-4. Para este grafo no existe, pues, una coloración posible con tres colores o menos y, por tanto, la solución de la figura 1.5 es óptima en el sentido de que usa el menor número posible de colores. Para el problema original, ningún semáforo puede tener menos de cuatro fases para la intersección de la figura 1.1.

Así pues, considérese el control de un semáforo basado en la figura 1.5, donde cada fase del control corresponde a un color. En cada fase, se permiten los giros indicados en la fila de la tabla que corresponde a ese color, y se prohíben los restantes. Este modelo utiliza el menor número posible de fases. □

Seudolenguaje y refinamiento por pasos

Una vez que se tiene un modelo matemático apropiado para un problema, puede formularse un algoritmo basado en ese modelo. Las versiones iniciales del algoritmo a menudo están mezcladas en proposiciones generales que deberán refinarse después en instrucciones más pequeñas y definidas. Por ejemplo, se ha descrito el algoritmo ávido de coloración de grafos con expresiones del tipo «elegir algún vértice no coloreado». Es de esperar que esas instrucciones sean lo bastante claras para com-

prender el razonamiento. Sin embargo, para convertir en programa un algoritmo tan informal, es necesario pasar por varias etapas de formalización (*refinamiento por pasos*) hasta llegar a un programa cuyos pasos tengan un significado formalmente definido en el manual de algún lenguaje de programación.

Ejemplo 1.2. Considérese el algoritmo ávido de coloración de grafos con el fin de convertirlo en un programa en Pascal. A continuación, se supondrá que existe un grafo G con algunos vértices que pueden colorearse. El siguiente programa *ávido* determina un conjunto de vértices llamado *nue_color* a los cuales se puede asignar un nuevo color. Se llama repetidas veces al programa hasta colorear todos los vértices. Sin entrar en detalles, se puede especificar *ávido* enseudolenguaje como se muestra en la figura 1.6.

```

procedure ávido ( var  $G$ : GRAFO; var nue_color: CONJUNTO );
    { ávido asigna a nue_color un conjunto de vértices de  $G$  a los que se
      puede dar el mismo color }
    begin
(1)      nue_color :=  $\emptyset$ ; †
(2)      for cada vértice  $v$  no coloreado de  $G$  do
(3)        if  $v$  no es adyacente a ningún vértice de nue_color then begin
(4)          marca  $v$  como coloreado;
(5)          agrega  $v$  a nue_color
        end
    end; { ávido }
  
```

Fig. 1.6. Primer refinamiento del algoritmo ávido.

En la figura 1.6 se observan algunas características importantes de esteseudolenguaje. La primera de ellas es que se utilizan letras minúsculas negritas para las palabras clave de Pascal que tienen el mismo significado que en Pascal estándar. Las palabras en mayúscula, como GRAFO y CONJUNTO ††, son nombres de «tipos de datos abstractos». Estos se definirán mediante declaraciones tipo Pascal, y las operaciones asociadas con esos tipos de datos abstractos se definirán mediante procedimientos de Pascal al crear el programa final. En las dos secciones siguientes se analizarán con más detalle los tipos de datos abstractos.

Las construcciones de flujo de control de Pascal como **if**, **for** y **while** se pueden utilizar en las proposiciones enseudolenguaje, pero las expresiones condicionales, como la de la línea (3), pueden ser proposiciones informales, en vez de expresiones condicionales de Pascal. Obsérvese que la asignación de la línea (1) utiliza una expresión informal a la derecha, y que el ciclo **for** de la línea (2) itera sobre un conjunto.

Para su ejecución, el programa enseudolenguaje de la figura 1.6 se debe refinar hasta convertirlo en un programa convencional en Pascal. Este proceso no se reali-

† El símbolo $\emptyset$ representa el conjunto vacío.

†† Se debe distinguir entre el tipo de datos abstracto CONJUNTO y el tipo incorporado set de Pascal.

zará en forma completa para el programa de este ejemplo, pero sí se dará un ejemplo de refinamiento, transformando la proposición *if* de la línea (3) de la figura 1.6 en un código más convencional.

Para probar si un vértice v es adyacente a otro vértice de *nue_color*, se puede considerar cada miembro w de *nue_color* y examinar el grafo G para determinar la existencia de una arista entre v y w . Una manera organizada de realizar esta prueba consiste en utilizar una variable booleana *encontrado* que indique si se ha encontrado una arista. De este modo, las líneas (3) a (5) de la figura 1.6 pueden reemplazarse por el código de la figura 1.7.

```

procedure ávido ( var  $G$ : GRAFO; var nue_color: CONJUNTO );
  begin
(1)    nue_color :=  $\emptyset$  ;
(2)    for cada vértice  $v$  no coloreado de  $G$  do begin
(3.1)      encontrado := false;
(3.2)      for cada vértice  $w$  de nue_color do
(3.3)        if hay una arista entre  $v$  y  $w$  en  $G$  then
(3.4)          encontrado := true;
(3.5)      if encontrado = false then begin
          {  $v$  no es adyacente a ningún vértice de nue_color }
(4)        marca  $v$  como coloreado;
(5)        agrega  $v$  a nue_color
          end
      end
  end;

```

Fig. 1.7. Refinamiento parcial de la figura 1.6.

Ahora, el algoritmo se ha reducido a una colección de operaciones sobre dos conjuntos de vértices. El ciclo externo, compuesto por las líneas (2) a (5), itera sobre el conjunto de vértices no coloreados de G . El ciclo interno, líneas (3.2) a (3.4); itera sobre los vértices que se encuentran en el conjunto *nue_color*. La línea (5) agrega vértices recién coloreados a *nue_color*.

En un lenguaje de programación como Pascal hay muchas formas de representar conjuntos. En los capítulos 4 y 5 se estudiarán varias de esas representaciones. En este ejemplo, los conjuntos de vértices se representarán simplemente por otro tipo de datos abstracto, LISTA, que se puede aplicar mediante una lista de enteros terminada con un valor especial *nulo* (para lo cual se podría aplicar el valor 0). Estos enteros podrían almacenarse, por ejemplo, en un arreglo, aunque hay muchas otras formas de representar una LISTA, como se verá en el capítulo 2.

Se puede ahora reemplazar la proposición **for** de la línea (3.2) por un ciclo en el que w se inicializa como primer miembro de *nue_color* y cambia al siguiente miembro en cada repetición del ciclo. También se puede realizar el mismo refinamiento con el ciclo **for** de la línea (2) de la figura 1.6. El procedimiento *ávido* revisado

se muestra en la figura 1.8. Después de esto, aún hay refinamientos por hacer, pero se hará una pausa para repasar lo estudiado. □

```

procedure ávido ( var G: GRAFO; var nue_color: LISTA );
  { ávido asigna a nue_color los vértices a los que se
    puede dar el mismo color }
  var
    encontrado: boolean;
    v, w: integer;
  begin
    nue_color := Ø;
    v := primer vértice no coloreado de G;
    while v <> nulo do begin
      encontrado := false;
      w := primer vértice de nue_color;
      while w <> nulo do begin
        if hay una arista entre v y w en G then
          encontrado := true;
          w := siguiente vértice de nue_color;
        end;
      if encontrado = false do begin
        marca v como coloreado;
        agrega v a nue_color;
      end;
      v := siguiente vértice no coloreado de G
    end
  end; { ávido }

```

Fig. 1.8. Procedimiento ávido refinado.

Resumen

En la figura 1.9 se representa el proceso de programación tal como se tratará en este libro. La primera etapa es la modelación, mediante un modelo matemático apropiado, como un grafo. En esta etapa, la solución del problema es un algoritmo expresado de manera muy informal.

En la siguiente etapa, el algoritmo se escribe enseudolenguaje, es decir, en una mezcla de construcciones de Pascal y proposiciones menos formales en español. Esta etapa se alcanza sustituyendo el español informal por secuencias de proposiciones cada vez más detalladas, en un proceso denominado *refinamiento por pasos*. En algún punto de este proceso, el programa enseudolenguaje estará suficientemente detallado para que las operaciones que se deban realizar con los distintos tipos de da-

tos estén bien determinadas. Entonces, se crean los tipos de datos abstractos para cada tipo de datos (con excepción de los datos de tipo elemental como los enteros, los reales o las cadenas de caracteres) dando un nombre de procedimiento a cada operación y sustituyendo los usos de las operaciones por invocaciones a los procedimientos correspondientes.

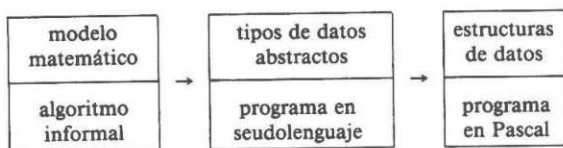


Fig. 1.9. Proceso de solución de problemas.

En la tercera etapa, se elige una aplicación para cada tipo de datos abstracto y se escribe el procedimiento que corresponde a cada operación con ese tipo. También se reemplaza por código Pascal toda proposición informal que quede en el algoritmo en pseudolenguaje. El resultado será un programa ejecutable. Después de depurarlo, será un programa operativo y se espera que por haber aplicado el desarrollo por pasos bosquejado en la figura 1.9, se requiera poca depuración.

1.2 Tipos de datos abstractos

Con seguridad, la mayor parte de los conceptos introducidos en la sección anterior le es familiar al lector desde sus primeros cursos de programación. Quizá la única noción nueva sea la de tipo de datos abstracto; por eso, antes de continuar, se analizará el papel de estos tipos durante el proceso general de diseño de programas. Para comenzar, es útil comparar este concepto con el más familiar, de procedimiento.

Los procedimientos, herramientas esenciales de programación, generalizan el concepto de operador. En vez de limitarse a los operadores incorporados de un lenguaje de programación (suma, resta, etcétera), con el uso de procedimientos, un programador es libre de definir sus propios operadores y aplicarlos a operandos que no tienen por qué ser de tipo fundamental. Un ejemplo de esta aplicación de los procedimientos es una rutina de multiplicación de matrices.

Otra ventaja de los procedimientos es que pueden utilizarse para *encapsular* partes de un algoritmo, localizando en una sección de un programa todas las proposiciones que tienen importancia en relación con cierto aspecto de éste. Un ejemplo de encapsulación es el uso de un procedimiento para leer todas las entradas y verificar su validez. La ventaja de realizar encapsulaciones es que se sabe hacia dónde ir para hacer cambios en el aspecto encapsulado del problema. Por ejemplo, si se decide verificar que todas las entradas sean no negativas, sólo se necesita cambiar unas cuantas líneas de código, y se sabe con exactitud dónde están esas líneas.

Definición de tipo de datos abstracto

Se puede pensar en un *tipo de datos abstracto* (TDA) como en un modelo matemático con una serie de operaciones definidas en ese modelo. Un ejemplo sencillo de TDA son los conjuntos de números enteros con las operaciones de unión, intersección y diferencia. Las operaciones de un TDA pueden tener como operandos no sólo los casos del TDA que se define, sino también otros tipos de operandos, como enteros o casos de otro TDA, y el resultado de una operación puede no ser un caso de ese TDA. Sin embargo, se supone que al menos un operando, o el resultado, de alguna operación pertenece al TDA en cuestión.

Las dos propiedades de los procedimientos mencionadas anteriormente, generalización y encapsulación, son igualmente aplicables a los tipos de datos abstractos. Los TDA son generalizaciones de los tipos de datos primitivos (enteros, reales, etcétera), al igual que los procedimientos son generalizaciones de operaciones primitivas (suma, resta, etcétera). Un TDA encapsula cierto tipo de datos en el sentido de que es posible localizar la definición del tipo y todas las operaciones con ese tipo se pueden localizar en una sección del programa. Así, si se desea cambiar la forma de implantar un TDA, se sabe hacia dónde dirigirse, y revisando una pequeña sección del programa se puede tener la seguridad de que no hay detalles en otras partes que puedan ocasionar errores relacionados con ese tipo de datos. Además, fuera de la sección en la que están definidas las operaciones con el TDA, éste se puede utilizar como si fuese un tipo de datos primitivo, es decir, sin preocuparse por la aplicación. Un inconveniente es que ciertas operaciones pueden implicar más de un TDA, y para que todo marche bien en tales casos, debe hacerse referencia a estas operaciones en las secciones de los dos TDA.

Para ilustrar las ideas básicas considérese el procedimiento *ávido* de la sección anterior (Fig. 1.8), implantado mediante operaciones primitivas en el tipo de datos abstracto LISTA (de enteros). Las operaciones ejecutadas en la LISTA *nue_color* fueron las siguientes:

1. vaciar la lista;
2. obtener el primer miembro de la lista y devolver *nulo* si la lista estaba vacía;
3. obtener el siguiente miembro de la lista y devolver *nulo* si no hay miembro siguiente, y, por último,
4. insertar un entero en la lista.

Existen muchas estructuras de datos que se pueden utilizar para implantar eficientemente a una lista como éstas; este tema se estudiará con profundidad en el capítulo 2. Si en la figura 1.8 se reemplazan esas operaciones por las proposiciones

1. ANULA (*nue_color*);
2. *w* := PRIMERO (*nue_color*);
3. *w* := SIGUIENTE (*nue_color*);
4. INSERTA (*v*, *nue_color*);

entonces se aprecia un aspecto importante de los tipos de datos abstractos. Se puede aplicar un tipo en la forma que se desee sin que los programas, como el de la fi-

gura 1.8, que utilizan objetos de ese tipo sufran cambios; solamente cambian los procedimientos que aplican las operaciones en el tipo.

Volviendo al tipo de datos abstracto GRAFO, se observa la necesidad de las siguientes operaciones:

1. obtener el primer vértice no coloreado;
2. determinar si hay una arista entre dos vértices;
3. marcar un vértice como coloreado, y
4. obtener el siguiente vértice no coloreado.

Es evidente que se necesitan otras operaciones fuera del procedimiento *ávido*, como insertar vértices y aristas en un grafo y marcar todos sus vértices como no coloreados. Hay muchas estructuras de datos que se pueden usar para apoyar grafos con estas operaciones; este tema se estudia en los capítulos 6 y 7.

Es importante resaltar el hecho de que no hay límite para el número de operaciones que se pueden aplicar a diversos casos de un modelo matemático dado. Cada conjunto de operaciones define un TDA distinto. Algunos ejemplos de operaciones que podrían definirse con un tipo de datos abstracto CONJUNTO son los siguientes:

1. ANULA(A). Este procedimiento hace que el valor del conjunto A sea el conjunto vacío.
2. UNION(A, B, C). Este procedimiento toma dos argumentos, A y B , cuyos valores son conjuntos, y hace que el conjunto C tome el valor de la unión de A y B .
3. TAMAÑO(A). Esta función toma un argumento A , cuyo valor es un conjunto, y devuelve un objeto de tipo entero cuyo valor es el número de elementos de A .

La *implantación* de un TDA es la traducción en proposiciones de un lenguaje de programación, de la declaración que define una variable como perteneciente a ese tipo, además de un procedimiento en ese lenguaje por cada operación del TDA. Una implantación elige una *estructura de datos* para representar el TDA; cada estructura de datos se construye a partir de los tipos de datos fundamentales del lenguaje de programación base, utilizando los dispositivos de estructuración de datos disponibles. Los arreglos y las estructuras de registro son dos importantes dispositivos de estructuración de datos de Pascal. Por ejemplo, una implantación posible de la variable S del tipo CONJUNTO sería un arreglo que contuviera los miembros de S .

Una razón importante para afirmar que dos TDA son diferentes si tienen el mismo modelo matemático, pero distintas operaciones, es que lo apropiado de una realización depende en gran medida de las operaciones que se van a realizar. Gran parte de este libro se dedica al examen de algunos modelos matemáticos básicos, como los conjuntos y los grafos, y al desarrollo de las realizaciones preferibles para varios conjuntos de operaciones.

Lo ideal sería escribir programas en lenguajes cuyos tipos de datos y operaciones primitivos estuvieran muy cerca de los modelos y operaciones de los TDA que se utilizarán aquí. En muchos sentidos, Pascal no es el lenguaje más apropiado para implantar varios TDA comunes, pero ninguno de los lenguajes en que un TDA puede declararse más directamente es tan conocido. Consúltense las notas bibliográficas del final del capítulo.

1.3 Tipos de datos, estructuras de datos y tipos de datos abstractos

Aunque los términos «tipo de datos» (o simplemente «tipo»), «estructura de datos» y «tipo de datos abstracto» parecen semejantes, su significado es diferente. En un lenguaje de programación, el *tipo de datos* de una variable es el conjunto de valores que ésta puede tomar. Por ejemplo, una variable de tipo booleano puede tomar los valores verdadero o falso, pero ningún otro. Los tipos de datos básicos varían de un lenguaje a otro; en Pascal son entero (*integer*), real (*real*), booleano (*boolean*) y carácter (*char*). Las reglas para construir tipos de datos compuestos a partir de los básicos también varían de un lenguaje a otro; se verá ahora la construcción de esos tipos en Pascal.

Un tipo de datos abstracto es un modelo matemático, junto con varias operaciones definidas sobre ese modelo. Tal como se indicó, en este libro se diseñarán los algoritmos en función de los TDA, pero para implantar un algoritmo en un lenguaje de programación determinado debe hallarse alguna manera de representar los TDA en función de los tipos de datos y los operadores manejados por ese lenguaje. Para representar el modelo matemático básico de un TDA, se emplean *estructuras de datos*, que son conjuntos de variables, quizá de tipos distintos, conectadas entre sí de diversas formas.

El componente básico de una estructura de datos es la *celda*. Se puede representar una celda como una caja capaz de almacenar un valor tomado de algún tipo de datos básico o compuesto. Las estructuras de datos se crean dando nombres a agregados de celdas y (opcionalmente) interpretando los valores de algunas celdas como representantes de conexiones entre celdas (por ejemplo, los apuntadores).

El mecanismo de agregación más sencillo en Pascal y en la mayor parte de los lenguajes de programación es el *arreglo* (unidimensional), que es una sucesión de celdas de un tipo dado al cual se llamará casi siempre «tipo_celda». Se puede imaginar un arreglo como una transformación del conjunto índice (como los enteros 1, 2, ..., *n*) al tipo tipo-celda. Se puede hacer referencia a una celda de un arreglo dando el nombre del arreglo en unión de un valor tomado del conjunto índice del arreglo. En Pascal, el conjunto índice puede ser un tipo enumerado definido por el programador, como (norte, sur, este, oeste), o un subrango como 1..10. Los valores de las celdas de un arreglo pueden ser de cualquier tipo. Así, la declaración

nombre: array [tipo_índice] of tipo_celda;

indica que *nombre* es una sucesión de celdas, una por cada valor del tipo tipo_índice; el contenido de las celdas puede ser cualquier miembro del tipo tipo_celda.

A propósito de esto, Pascal tiene una riqueza considerable de tipos índice. Muchos lenguajes sólo admiten subrangos (conjuntos finitos de enteros consecutivos) como tipos índice. Por ejemplo, en FORTRAN, para indizar un arreglo con letras, debe simularse el efecto utilizando índices enteros, como el índice 1 para representar la 'A', el 2 para la 'B', y así sucesivamente.

Otro mecanismo habitual para agrupar celdas en los lenguajes de programación es la *estructura de registro*. Un *registro* es una celda constituida por un conjunto de celdas llamadas *campos*, que pueden ser de tipos distintos. A menudo, los registros

se agrupan en arreglos; el tipo definido por la agregación de los campos de un registro se convierte en el tipo_celda del arreglo. Por ejemplo, la declaración en Pascal

```
var
  lista_reg: array [1..4] of record
    datos: real;
    siguiente: integer
  end
```

declara que *lista_reg* es un arreglo de cuatro elementos cuyas celdas son registros que tienen dos campos: *datos* y *siguiente*.

Un tercer método de agrupación que se encuentra en Pascal y en otros lenguajes es el *archivo* (*file*). Al igual que un arreglo unidimensional, un archivo es una sucesión de valores de un tipo particular. La diferencia está en que un archivo no tiene tipo índice; los elementos sólo son accesibles en el orden en que aparecen en el archivo. En contraste, tanto el arreglo como el registro son estructuras de «acceso aleatorio», lo cual significa que el tiempo necesario para acceder a un componente de un arreglo o de un registro es independiente del valor del índice del arreglo o del selector de campo. La ventaja de agrupar por archivos en lugar de hacerlo por arreglos es que el número de elementos de un archivo puede ser ilimitado y variable con el tiempo.

Apuntadores y cursores

Además de las características de agrupación-celdas de un lenguaje de programación, es posible representar relaciones entre celdas mediante apuntadores y cursores. Un *apuntador* es una celda cuyo valor indica o señala a otra. Cuando se representan gráficamente estructuras de datos, el hecho de que una celda *A* sea un apuntador a la celda *B* se indica con una flecha de *A* a *B*.

En Pascal, se puede crear una variable *ap* que apunte a celdas de un tipo determinado, como tipo_celda, mediante la declaración

```
var
  ap: ↑ tipo_celda
```

En Pascal se utiliza una flecha ascendente postfija como operador de desreferenciación. Así, la expresión *ap*↑ denota el valor (de tipo tipo_celda) de la celda a la que apunta *ap*.

Un *cursor* es una celda de valor entero que se utiliza como apuntador a un arreglo. Como métodos de conexión, cursores y apuntadores son en esencia lo mismo, con la diferencia de que un cursor se puede utilizar en lenguajes que, como FORTRAN, carecen de los tipos apuntadores explícitos que tiene Pascal. Al tratar una celda de tipo entero como un valor índice para un arreglo, se hace que esa celda apunte a una celda del arreglo. Desafortunadamente, esta técnica sólo sirve para apuntar a celdas contenidas en un arreglo; no hay manera razonable de interpretar un entero como un «apuntador» a una celda que no sea parte de un arreglo.

En este texto, se dibujará una flecha desde una celda cursor hacia la celda a la

que «apunta». En ocasiones, se mostrará también el entero de la celda cursor, para recordar que no se trata de un verdadero apuntador. El lector debe observar que el mecanismo apuntador de Pascal es tal que a las celdas de los arreglos sólo se puede «apuntar» con cursores, y no con verdaderos apuntadores. Ciertos lenguajes, como PL/I o C, permiten que los componentes de un arreglo sean apuntados tanto por cursores como por apuntadores, mientras que en otros, como FORTRAN o ALGOL, no existe el tipo apuntador y sólo es posible usar cursores.

Ejemplo 1.3. En la figura 1.10 se observa una estructura de datos con dos partes, constituida por una cadena de celdas que contienen cursores al arreglo *lista_reg* definido antes. El propósito del campo *siguiente* de *lista_reg* es apuntar a otro registro del arreglo. Por ejemplo, *lista_reg* [4]. *siguiente* es 1, por tanto, el registro 4 va seguido del registro 1. Si se supone que el registro 4 es el primero, el campo *siguiente* de *lista_reg* ordena los registros en la secuencia 4, 1, 3, 2. Obsérvese que el campo *siguiente* del registro 2 es 0, lo cual indica que no existe un registro que lo siga. Una convención útil que se adoptará en este libro consiste en usar 0 como un «apuntador NIL» cuando se trabaje con cursores. Esto sólo da buenos resultados si también se adopta la convención de que los arreglos a los que «apuntan» cursores han de indexarse a partir de 1, no de 0.

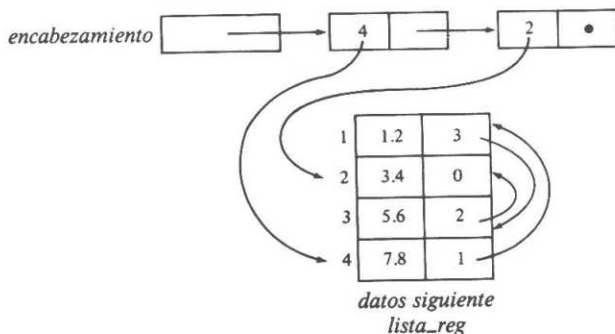


Fig. 1.10. Ejemplo de una estructura de datos.

Las celdas de la cadena de registros de la figura 1.10 son del tipo

```
type
  tipo_reg = record
    cursor: integer;
    ap: ↑ tipo_reg
  end
```

A la cadena apunta una variable llamada *encabezamiento*, que es de tipo $\uparrow$ tipo_reg; *encabezamiento* apunta a un registro anónimo de tipo tipo_reg $\uparrow$. Ese registro tiene valor 4 en el campo *cursor*; este 4 se considera como un índice del arreglo *lista_reg*, y en el campo *ap* tiene un verdadero apuntador a otro registro anónimo. Este último tiene un índice en su campo *cursor*, que indica la posición 2 en *lista_reg*, y un apuntador *nil* en su campo *ap*. □

1.4 Tiempo de ejecución de un programa

Cuando se resuelve un problema, con frecuencia hay necesidad de elegir entre varios algoritmos. ¿Cómo se debe elegir? Hay dos objetivos que suelen contradecirse:

1. Que el algoritmo sea fácil de entender, codificar y depurar.
2. Que el algoritmo use eficientemente los recursos del computador y, en especial, que se ejecute con la mayor rapidez posible.

Cuando se escribe un programa que se va a usar una o pocas veces, el primer objetivo es el más importante. En tal caso, es muy probable que el costo del tiempo de programación exceda en mucho al costo de ejecución del programa, de modo que el costo a optimar es el de escritura del programa. En cambio, cuando se presenta un problema cuya solución se va a utilizar muchas veces, el costo de ejecución del programa puede superar en mucho al de escritura, en especial si en la mayor parte de las ejecuciones se dan entradas de gran tamaño. Entonces, es más ventajoso, desde el punto de vista económico, realizar un algoritmo complejo siempre que el tiempo de ejecución del programa resultante sea significativamente menor que el de un programa más evidente. Y aun en situaciones como esa, quizá sea conveniente implantar primero un algoritmo simple, con el objeto de determinar el beneficio real que se obtendría escribiendo un programa más complicado. En la construcción de un sistema complejo, a menudo es deseable implantar un prototipo sencillo en el cual se puedan efectuar simulaciones y mediciones antes de dedicarse al diseño definitivo. De esto se concluye que un programador no sólo debe estar al tanto de las formas de lograr que un programa se ejecute con rapidez, sino que también debe saber cuándo aplicar esas técnicas y cuándo ignorarlas.

Medición del tiempo de ejecución de un programa

El tiempo de ejecución de un programa depende de factores como:

1. los datos de entrada al programa,
2. la calidad del código generado por el compilador utilizado para crear el programa objeto,

$\uparrow$ El registro no tiene nombre conocido porque se creó con una llamada *new* (*encabezamiento*) cuyo efecto fue que *encabezamiento* apuntara a ese registro recién creado. No obstante, en el interior de la máquina existe una dirección de memoria que puede emplearse para localizar la celda.

3. la naturaleza y rapidez de las instrucciones de máquina empleadas en la ejecución del programa, y
4. la complejidad de tiempo del algoritmo base del programa.

El hecho de que el tiempo de ejecución dependa de la entrada, indica que el tiempo de ejecución de un programa debe definirse como una función de la entrada. Con frecuencia, el tiempo de ejecución no depende de la entrada exacta, sino sólo de su «tamaño». Un buen ejemplo de esto es el proceso conocido como *clasificación* (*sorting*), que se analizará en el capítulo 8. En un problema de clasificación, se da como entrada una lista de elementos para ordenar a fin de producir como salida otra lista con los mismos elementos, pero clasificados de menor a mayor o viceversa. Por ejemplo, dada la lista 2, 1, 3, 1, 5, 8 como entrada, se desea producir la lista 1, 1, 2, 3, 5, 8 como salida. Se dice entonces que los elementos de la segunda lista están en *orden de menor a mayor*. La medida natural del tamaño de la entrada a un programa de clasificación es el número de elementos a ordenar o, en otras palabras, la longitud de la lista de entrada. En general, la longitud de la entrada es una medida apropiada de tamaño, y se supondrá que tal es la medida utilizada a menos que se especifique lo contrario.

Se acostumbra, pues, a denominar $T(n)$ al tiempo de ejecución de un programa con una entrada de tamaño n . Por ejemplo, algunos programas pueden tener un tiempo de ejecución $T(n) = cn^2$, donde c es una constante. Las unidades de $T(n)$ se dejan sin especificar, pero se puede considerar a $T(n)$ como el número de instrucciones ejecutadas en un computador idealizado.

Para muchos programas, el tiempo de ejecución es en realidad una función de la entrada específica, y no sólo del tamaño de ella. En este caso se define $T(n)$ como el tiempo de ejecución del *peor caso*, es decir, el máximo valor del tiempo de ejecución para entradas de tamaño n . También suele considerarse $T_{prom}(n)$, el valor medio del tiempo de ejecución de todas las entradas de tamaño n . Aunque $T_{prom}(n)$ parece una medida más razonable, a menudo es engañoso suponer que todas las entradas son igualmente probables. En la práctica, casi siempre es más difícil determinar el tiempo de ejecución promedio que el del peor caso, pues el análisis se hace intratable en matemáticas, y la noción de entrada «promedio» puede carecer de un significado claro. Así pues, se utilizará el tiempo de ejecución del peor caso como medida principal de la complejidad de tiempo, aunque se mencionará la complejidad del caso promedio cuando pueda hacerse en forma significativa.

Considérense ahora las observaciones 2 y 3 anteriores: a saber, que el tiempo de ejecución depende del compilador y de la máquina utilizados. Este hecho implica que no es posible expresar $T(n)$ en unidades estándares de tiempo, como segundos. Antes bien, sólo se pueden hacer observaciones como «el tiempo de ejecución de tal algoritmo es proporcional a n^2 », sin especificar la constante de proporcionalidad, pues depende en gran medida del compilador, la máquina y otros factores.

Notación asintótica («o mayúscula» y «omega mayúscula»)

Para hacer referencia a la velocidad de crecimiento de los valores de una función se usará la notación conocida como *notación asintótica* («o mayúscula»). Por ejemplo, de-

cir que el tiempo de ejecución $T(n)$ de un programa es $O(n^2)$, que se lee «o mayúscula de n al cuadrado» o tan sólo «o de n al cuadrado», significa que existen constantes enteras positivas c y n_0 tales que para n mayor o igual que n_0 , se tiene que $T(n) \leq cn^2$.

Ejemplo 1.4. Supóngase que $T(0) = 1$, $T(1) = 4$, y en general $T(n) = (n+1)^2$. Entonces se observa que $T(n)$ es $O(n^2)$ cuando $n_0 = 1$ y $c = 4$; es decir, para $n \geq 1$, se tiene que $(n+1)^2 \leq 4n^2$, que es fácil de demostrar. Obsérvese que no se puede hacer $n_0 = 0$, pues $T(0) = 1$ no es menor que $c0^2 = 0$ para ninguna constante c . $\square$

A continuación, se supondrá que todas las funciones del tiempo de ejecución están definidas en los enteros no negativos, y que sus valores son siempre no negativos, pero no necesariamente enteros. Se dice que $T(n)$ es $O(f(n))$ si existen constantes positivas c y n_0 tales que $T(n) \leq cf(n)$ cuando $n \geq n_0$. Cuando el tiempo de ejecución de un programa es $O(f(n))$, se dice que tiene *velocidad de crecimiento* $f(n)$.

Ejemplo 1.5. La función $T(n) = 3n^3 + 2n^2$ es $O(n^3)$. Para comprobar esto, sean $n_0 = 0$ y $c = 5$. Entonces, el lector puede mostrar que para $n \geq 0$, $3n^3 + 2n^2 \leq 5n^3$. También se podría decir que $T(n)$ es $O(n^4)$, pero sería una aseveración más débil que decir que $T(n)$ es $O(n^3)$.

A manera de segundo ejemplo, se demostrará que la función 3^n no es $O(2^n)$. Para esto, supóngase que existen constantes n_0 y c tales que para todo $n \geq n_0$, se tiene que $3^n \leq c2^n$. Entonces, $c \geq (3/2)^n$ para cualquier valor de $n \geq n_0$. Pero $(3/2)^n$ se hace arbitrariamente grande conforme n crece y, por tanto, ninguna constante c puede ser mayor que $(3/2)^n$ para toda n . $\square$

Cuando se dice que $T(n)$ es $O(f(n))$, se sabe que $f(n)$ es una cota superior para la velocidad de crecimiento de $T(n)$. Para especificar una cota inferior para la velocidad de crecimiento de $T(n)$, se usa la notación $T(n)$ es $\Omega(g(n))$, que se lee « $T(n)$ es omega mayúscula de $g(n)$ » o simplemente « $T(n)$ es omega de $g(n)$ », lo cual significa que existe una constante c tal que $T(n) \geq cg(n)$ para un número infinito de valores de n $\dagger$.

Ejemplo 1.6. Para verificar que la función $T(n) = n^3 + 2n^2$ es $\Omega(n^3)$, sea $c = 1$. Entonces, $T(n) \geq cn^3$ para $n = 0, 1, \dots$

En otro ejemplo, sea $T(n) = n$ para $n \geq 1$ impar, y sea $T(n) = n^2/100$ para $n \geq 0$ par. Para verificar que $T(n)$ es $\Omega(n^2)$, sea $c = 1/100$ y considérese el conjunto infinito de valores de n : $n = 0, 2, 4, 6, \dots$ $\square$

La «tiranía» de la velocidad de crecimiento

Se supondrá que es posible evaluar programas comparando sus funciones de tiempo de ejecución sin considerar las constantes de proporcionalidad. Según este supuesto, un programa con tiempo de ejecución $O(n^2)$ es mejor que uno con tiempo de ejecu-

$\dagger$ Obsérvese la asimetría existente entre las notaciones «o mayúscula» y «omega mayúscula». La razón de que esta asimetría sea con frecuencia útil, es que muchas veces un algoritmo es rápido con muchas entradas pero no con todas. Por ejemplo, para probar si la entrada es de longitud prima, existen algoritmos que se ejecutan muy rápido si la longitud es par, de modo que para el tiempo de ejecución no es posible obtener una buena cota inferior que sea válida para toda $n \geq n_0$.

ción $O(n^3)$ por ejemplo. Sin embargo, además de los factores constantes debidos al compilador y a la máquina, existe un factor constante debido a la naturaleza del programa mismo. Es posible, por ejemplo, que con una combinación determinada de compilador y máquina, el primer programa tarde $100n^2$ milisegundos, mientras el segundo tarda $5n^3$ milisegundos. En este caso, ¿no es preferible el segundo programa al primero?

La respuesta a esto depende del tamaño de las entradas que se espera que procesen los programas. Para entradas de tamaño $n < 20$, el programa con tiempo de ejecución $5n^3$ será más rápido que el de tiempo de ejecución $100n^2$. Así pues, si el programa se va a ejecutar principalmente con entradas pequeñas, será preferible el programa cuyo tiempo de ejecución es $O(n^3)$. No obstante, conforme n crece, la razón de los tiempos de ejecución, que es $5n^3/100n^2 = n/20$, se hace arbitrariamente grande. Así, a medida que crece el tamaño de la entrada, el programa $O(n^3)$ requiere un tiempo significativamente mayor que el programa $O(n^2)$. Pero si hay algunas entradas grandes en los problemas para cuya solución se están diseñando estos dos programas, será mejor optar por el programa cuyo tiempo de ejecución tiene la menor velocidad de crecimiento.

Otro motivo para al menos considerar programas cuyas velocidades de crecimiento sean lo más pequeñas posible, es que básicamente es la velocidad del crecimiento quien determina el tamaño de problema que se puede resolver en un computador. Para decirlo de otro modo, conforme los computadores se hacen más veloces, también aumentan los deseos del usuario de resolver con ellos problemas más grandes; sin embargo, a menos que un programa tenga una velocidad de crecimiento baja, como $O(n)$ u $O(n \log n)$, un incremento modesto en la rapidez de un computador no influye gran cosa en el tamaño del problema más grande que es posible resolver en una cantidad fija de tiempo.

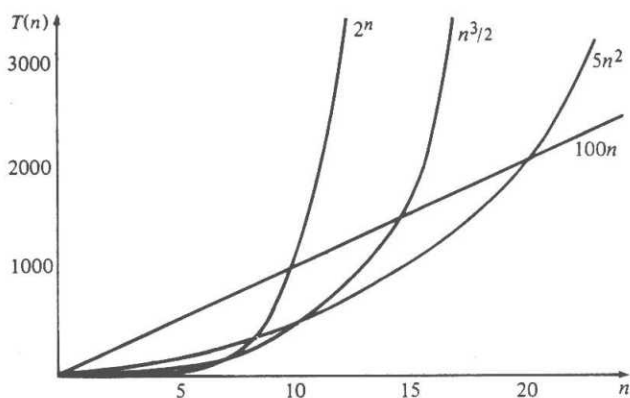


Fig. 1.11. Tiempos de ejecución de cuatro programas.

Ejemplo 1.7. En la figura 1.11 se pueden ver los tiempos de ejecución de cuatro programas de distintas complejidades de tiempo, medidas en segundos, para una combinación determinada de compilador y máquina. Supóngase que se dispone de 1000 segundos, o alrededor de 17 minutos, para resolver un problema determinado. ¿Qué tamaño de problema se puede resolver? Como se puede ver en la segunda columna de la figura 1.12, los cuatro algoritmos pueden resolver problemas de un tamaño similar en 10^3 segundos.

Supóngase ahora que se adquiere una máquina que funciona diez veces más rápido sin costo adicional. Entonces, con el mismo costo, es posible dedicar 10^4 segundos a la solución de problemas que antes requerían 10^3 segundos. El tamaño máximo de problema que es posible resolver ahora con cada uno de los cuatro programas se muestra en la tercera columna de la figura 1.12, y la razón entre los valores de las columnas segunda y tercera se muestra en la cuarta. Se observa que un aumento del 1000% en la velocidad del computador origina apenas un incremento del 30% en el tamaño de problema que se puede resolver con el programa $O(2^n)$. Los aumentos adicionales de un factor de diez en la rapidez del computador a partir de este punto originan aumentos porcentuales aún menores en el tamaño de los problemas. De hecho, el programa $O(2^n)$ sólo puede resolver problemas pequeños, independientemente de la rapidez del computador.

Tiempo de ejecución $T(n)$	Tamaño máximo de problema para 10^3 seg	Tamaño máximo de problema para 10^4 seg	Incremento en el tamaño máximo de problema
$100n$	10	100	10.0
$5n^2$	14	45	3.2
$n^3/2$	12	27	2.3
2^n	10	13	1.3

Fig. 1.12. Efecto de multiplicar por diez la velocidad del computador.

En la tercera columna de la figura 1.12 se puede apreciar una superioridad evidente del programa $O(n)$; éste permite un aumento del 1000% en el tamaño de problema para un incremento del 1000% en la rapidez del computador. Se observa que los programas $O(n^3)$ y $O(n^2)$ permiten aumentos de 230% y 320%, respectivamente, en el tamaño de problema, para un incremento del 1000% en la rapidez del computador. Estas razones se mantendrán vigentes para incrementos adicionales en la rapidez del computador. □

Mientras exista la necesidad de resolver problemas cada vez más grandes, se producirá una situación casi paradójica. A medida que los computadores aumenten su rapidez y disminuyan su precio, como con toda seguridad seguirá sucediendo, también el deseo de resolver problemas más grandes y complejos seguirá creciendo. Así, la importancia del descubrimiento y el empleo de algoritmos eficientes (aquellos cuyas velocidades de crecimiento sean pequeñas) irá en aumento, en lugar de disminuir.

Aspectos importantes

Es necesario subrayar de nuevo que la velocidad de crecimiento del tiempo de ejecución del peor caso no es el único criterio, ni necesariamente el más importante, para evaluar un algoritmo o un programa. A continuación se presentan algunas condiciones en las cuales el tiempo de ejecución de un programa se puede ignorar en favor de otros factores.

1. Si un programa se va a utilizar sólo algunas veces, el costo de su escritura y depuración es el dominante, de manera que el tiempo de ejecución raramente influirá en el costo total. En ese caso debe elegirse el algoritmo que sea más fácil de aplicar correctamente.
2. Si un programa se va a ejecutar sólo con entradas «pequeñas», la velocidad de crecimiento del tiempo de ejecución puede ser menos importante que el factor constante de la fórmula del tiempo de ejecución. Determinar qué es una entrada «pequeña», depende de los tiempos de ejecución exactos de los algoritmos implicados. Hay algoritmos (como el de multiplicación de enteros de Schonhage y Strassen [1971]) que son asintóticamente los más eficientes para sus problemas, pero nunca se han llevado a la práctica ni siquiera con los problemas más grandes, debido a que la constante de proporcionalidad es demasiado grande comparada con la de otros algoritmos menos «eficientes».
3. Un algoritmo eficiente pero complicado puede no ser apropiado porque posteriormente puede tener que darle mantenimiento otra persona distinta del escritor. Se espera que al difundir el conocimiento de las principales técnicas de diseño de algoritmos eficientes, se podrán utilizar libremente algoritmos más complejos, pero debe considerarse la posibilidad de que un programa resulte inútil debido a que nadie entiende sus sutiles y eficientes algoritmos.
4. Existen ejemplos de algoritmos eficientes que ocupan demasiado espacio para ser aplicados sin almacenamiento secundario lento, lo cual puede anular la eficiencia.
5. En los algoritmos numéricos, la precisión y la estabilidad son tan importantes como la eficiencia.

1.5 Cálculo del tiempo de ejecución de un programa

Calcular el tiempo de ejecución de un programa arbitrario, aunque sólo sea una aproximación a un factor constante, puede ser un problema matemático complejo. Sin embargo, en la práctica esto suele ser más sencillo; basta con aplicar unos cuantos principios básicos. Antes de presentar estos principios, es importante aprender a sumar y a multiplicar en notación asintótica.

Supóngase que $T_1(n)$ y $T_2(n)$ son los tiempos de ejecución de dos fragmentos de programa P_1 y P_2 , y que $T_1(n)$ es $O(f(n))$ y $T_2(n)$ es $O(g(n))$. Entonces $T_1(n) + T_2(n)$, el tiempo de ejecución de P_1 seguido de P_2 , es $O(\max(f(n), g(n)))$. Para saber por qué, obsérvese que para algunas constantes, c_1, c_2, n_1 y n_2 , si $n \geq n_1$, entonces

$T_1(n) \leq c_1 f(n)$, y si $n \geq n_2$, entonces $T_2(n) \leq c_2 g(n)$. Sea $n_0 = \max(n_1, n_2)$. Si $n \geq n_0$, entonces $T_1(n) + T_2(n) \leq c_1 f(n) + c_2 g(n)$. De aquí se concluye que si $n \geq n_0$, entonces $T_1(n) + T_2(n) \leq (c_1 + c_2) \max(f(n), g(n))$; por tanto, $T_1(n) + T_2(n)$ es $O(\max(f(n), g(n)))$.

Ejemplo 1.8. La regla de la suma anterior puede usarse para calcular el tiempo de ejecución de una secuencia de pasos de programa, donde cada paso puede ser un fragmento de programa arbitrario con ciclos y ramificaciones. Supóngase que se tienen tres pasos cuyos tiempos de ejecución son, respectivamente, $O(n^2)$, $O(n^3)$ y $O(n \log n)$. Entonces, el tiempo de ejecución de los dos primeros pasos ejecutados en secuencia es $O(\max(n^2, n^3))$ que es $O(n^3)$. El tiempo de ejecución de los tres juntos es $O(\max(n^3, n \log n))$, que es $O(n^3)$. $\square$

En general, el tiempo de ejecución de una secuencia fija de pasos, dentro de un factor constante, es igual al tiempo de ejecución del paso con mayor tiempo de ejecución. En raras ocasiones dos pasos pueden tener tiempos de ejecución *inconmensurables* (ninguno es mayor que el otro, ni son iguales). Por ejemplo, puede haber pasos con tiempos de ejecución $O(f(n))$ y $O(g(n))$, donde

$$f(n) = \begin{cases} n^4 & \text{si } n \text{ es par} \\ n^2 & \text{si } n \text{ es impar} \end{cases} \quad g(n) = \begin{cases} n^2 & \text{si } n \text{ es par} \\ n^3 & \text{si } n \text{ es impar} \end{cases}$$

En tales casos, la regla de la suma debe aplicarse directamente; en el ejemplo, el tiempo de ejecución es $O(\max(f(n), g(n)))$, esto es, n^4 si n es par y n^3 si es impar.

Otra observación útil sobre la regla de la suma es que si $g(n) \leq f(n)$ para toda n mayor que una constante n_0 , entonces $O(f(n) + g(n))$ es lo mismo que $O(f(n))$. Por ejemplo, $O(n^2 + n)$ es lo mismo que $O(n^2)$.

La regla del producto es la siguiente: si $T_1(n)$ y $T_2(n)$ son $O(f(n))$ y $O(g(n))$, respectivamente, entonces $T_1(n)T_2(n)$ es $O(f(n)g(n))$. Se aconseja probar este hecho con las mismas ideas que se utilizaron para probar la regla de la suma. Según la regla del producto, $O(cf(n))$ significa lo mismo que $O(f(n))$ si c es una constante positiva cualquiera. Por ejemplo, $O(n^2/2)$ es lo mismo que $O(n^2)$.

Antes de pasar a las reglas generales de análisis de los tiempos de ejecución de los programas, se presentará un ejemplo sencillo para proporcionar una visión general del proceso.

Ejemplo 1.9. Considérese el programa de clasificación *burbuja* de la figura 1.13, que ordena un arreglo de enteros de menor a mayor. El efecto total de cada recorrido del ciclo interno de las proposiciones (3) a (6), es hacer que el menor de los elementos suba hasta el principio del arreglo.

El número n de elementos que se van a clasificar es la medida apropiada del tamaño de la entrada. La primera observación que se hace es que cada proposición de asignación toma cierta cantidad constante de tiempo, independiente del tamaño de la entrada. Esto significa que las proposiciones (4), (5) y (6) toman tiempo $O(1)$ cada una. Obsérvese que $O(1)$ es la notación «o mayúscula» de una «cantidad constante». Por la regla de la suma, el tiempo de ejecución combinado de este grupo de proposiciones es $O(\max(1, 1, 1)) = O(1)$.

Ahora deben tenerse en cuenta las proposiciones condicionales y las de control de ciclos. Las proposiciones **if** y **for** están anidadas unas dentro de otras, de modo

```

procedure burbuja ( var A: array [1..n] of integer );
    { burbuja clasifica el arreglo A de menor a mayor }
var
    i, j, temp: integer;
begin
(1)       for i := 1 to n-1 do
(2)           for j := n downto i + 1 do
(3)               if A[j-1] > A[j] then begin
                    { intercambia A[j-1] y A[j] }
(4)                   temp := A[j-1];
(5)                   A[j-1] := A[j];
(6)                   A[j] := temp
                    end
end; { burbuja }
    
```

Fig. 1.13. Clasificación burbuja.

que se debe ir de dentro hacia fuera para obtener el tiempo de ejecución del grupo condicional y de cada ciclo. En cuanto a la proposición **if**, la prueba de la condición requiere tiempo $O(1)$. No se sabe si el cuerpo de la proposición **if** (líneas (4) a (6)) se ejecutará, pero dado que se busca el tiempo de ejecución del peor caso, se supone lo peor, esto es, que se ejecute. Por tanto, el grupo **if** de las proposiciones (3) a (6) requiere tiempo $O(1)$.

Así, siguiendo hacia fuera, se llega al ciclo **for** de las líneas (2) a (6). La regla general para un ciclo es que el tiempo de ejecución total resulta de sumar, en cada iteración, los tiempos empleados en ejecutar el cuerpo del ciclo en esa iteración. Debe acumularse al menos $O(1)$ por cada iteración para justificar el incremento del índice, con el fin de verificar si se alcanzó el límite y para saltar de vuelta al principio del ciclo. Para el ciclo de las líneas (2) a (6), el cuerpo tarda tiempo $O(1)$ en cada iteración. El número de iteraciones es $n-i$, de modo que, por la regla del producto, el total de tiempo invertido en el ciclo de las líneas (2) a (6) es $O((n-i) \times 1)$, o sea, $O(n-i)$.

Se sigue ahora con el ciclo externo, el cual contiene todas las proposiciones ejecutables del programa. La proposición (1) se ejecuta $n-1$ veces, de manera que el tiempo total de ejecución del programa tiene como cota superior una constante multiplicada por

$$\sum_{i=1}^{n-1} (n-i) = n(n-1)/2 = n^2/2 - n/2$$

que es $O(n^2)$. Por tanto, para ejecutar el programa de la figura 1.13 se necesita un tiempo proporcional al cuadrado del número de elementos que se van a clasificar.

En el capítulo 8 se presentarán programas de clasificación cuyo tiempo de ejecución es $O(n \log n)$, que es considerablemente menor, dado que $\log n$ † es mucho menor que n para valores grandes de n . □

Antes de proseguir con algunas reglas generales de análisis, recuérdese que la determinación de una cota superior precisa para el tiempo de ejecución de un programa, unas veces es sencilla, pero otras puede ser un desafío intelectual profundo. No existe un conjunto completo de reglas para analizar programas; en este libro sólo se proporcionarán ciertas sugerencias e ilustrarán algunos de sus aspectos más sutiles mediante ejemplos.

Se enumerarán ahora algunas reglas generales para el análisis de programas. En general, el tiempo de ejecución de una proposición o de un grupo de ellas puede tener como parámetros el tamaño de la entrada, una o más variables, o ambas cosas. El único parámetro permisible para el tiempo de ejecución del programa completo es n , el tamaño de la entrada.

1. El tiempo de ejecución de cada proposición de asignación (lectura y escritura), por lo común puede tomarse como $O(1)$. Hay unas cuantas excepciones, como en PL/I, donde una asignación puede implicar matrices arbitrariamente grandes, y en cualquier lenguaje donde se permitan llamadas a funciones en las proposiciones de asignación.
2. El tiempo de ejecución de una secuencia de proposiciones se determina por la regla de la suma. Esto es, el tiempo de ejecución de una secuencia es, dentro de un factor constante, el máximo tiempo de ejecución de una proposición de la secuencia.
3. El tiempo de ejecución de una proposición condicional *if* es el costo de las proposiciones que se ejecutan condicionalmente, más el tiempo para evaluar la condición. El tiempo para evaluar la condición, por lo general, es $O(1)$. El tiempo para una construcción *if-then-else* es la suma del tiempo requerido para evaluar la condición más el mayor entre los tiempos necesarios para ejecutar las proposiciones cuando la condición es verdadera y el tiempo de ejecución de las proposiciones cuando la condición es falsa.
4. El tiempo para ejecutar un ciclo es la suma, sobre todas las iteraciones del ciclo, del tiempo de ejecución del cuerpo y del empleado para evaluar la condición de terminación (este último suele ser $O(1)$). A menudo este tiempo es, despreciando factores constantes, el producto del número de iteraciones del ciclo y el mayor tiempo posible para una ejecución del cuerpo, pero, por seguridad, debe considerarse cada iteración por separado. Por lo común, se conoce con certeza el número de iteraciones, pero en ocasiones no es posible determinarlo con precisión. Incluso puede ocurrir que un programa no sea un algoritmo y que no exista un límite al número de iteraciones de ciertos ciclos.

† A menos que se especifique lo contrario, todos los logaritmos son de base 2. Obsérvese que $O(\log n)$ no depende de la base del logaritmo, puesto que $\log_a n = c \log_b n$, donde $c = \log_a b$.

Llamadas a procedimientos

Si se tiene un programa con procedimientos que no son recursivos, es posible calcular el tiempo de ejecución de los distintos procedimientos, uno a la vez, partiendo de aquellos que no llaman a otros. (Recuérdese que la invocación a una función debe considerarse una «llamada».) Debe haber al menos un procedimiento con esa característica, a menos que como mínimo un procedimiento sea recursivo. Después, puede evaluarse el tiempo de ejecución de los procedimientos que sólo llaman a procedimientos que no hacen llamadas, usando los tiempos de ejecución de los procedimientos llamados evaluados antes. Se continúa el proceso evaluando el tiempo de ejecución de cada procedimiento después de haber evaluado los tiempos correspondientes a los procedimientos que llama.

Si hay procedimientos recursivos, no es posible ordenar las evaluaciones de modo que cada una utilice sólo evaluaciones ya realizadas. Lo que se debe hacer ahora es asociar a cada procedimiento recursivo una función de tiempo desconocida $T(n)$, donde n mide el tamaño de los argumentos del procedimiento. Luego se puede obtener una *recurrencia* para $T(n)$, es decir, una ecuación para $T(n)$ en función de $T(k)$ para varios valores de k .

Se conocen ya técnicas para resolver varias clases de recurrencias; algunas de ellas se presentarán en el capítulo 9. Aquí se mostrará cómo analizar un programa recursivo sencillo.

Ejemplo 1.10. La figura 1.14 muestra un programa recursivo para calcular $n!$, que es el producto de todos los enteros de 1 a n inclusive.

Una medida de tamaño apropiado para esta función es el valor de n . Sea $T(n)$ el tiempo de ejecución para $fact(n)$. El tiempo de ejecución para las líneas (1) y (2) es $O(1)$, y para la línea (3) es $O(1) + T(n-1)$. Por tanto, para ciertas constantes c y d ,

$$T(n) = \begin{cases} c + T(n-1) & \text{si } n > 1 \\ d & \text{si } n \leq 1 \end{cases} \quad (1.1)$$

```
function fact ( n: integer ) : integer;
    { fact(n) calcula n! }
begin
(1)     if n <= 1 then
(2)         fact := 1
        else
(3)         fact := n * fact(n-1)
end; { fact }
```

Fig. 1.14. Programa recursivo para calcular factoriales.

Suponiendo que $n > 2$, se puede desarrollar $T(n-1)$ en (1.1) para obtener

$$T(n) = 2c + T(n-2) \text{ si } n > 2$$

Esto es, $T(n-1) = c + T(n-2)$, como se puede ver al sustituir n por $n-1$ en (1.1). Así pues, es posible reemplazar $T(n-1)$ con $c + T(n-2)$ en la ecuación $T(n) = c + T(n-1)$. Después, se puede usar (1.1) para desarrollar $T(n-2)$, con lo que se obtiene

$$T(n) = 3c + T(n-3) \quad \text{si } n > 3$$

y así sucesivamente. En general,

$$T(n) = ic + T(n-i) \quad \text{si } n > i$$

Por último, cuando $i = n-1$, se obtiene

$$T(n) = c(n-1) + T(1) = c(n-1) + d \quad (1.2)$$

Por (1.2) se concluye que $T(n)$ es $O(n)$. Es importante observar que en este análisis se ha supuesto que la multiplicación de dos enteros es una operación de tiempo $O(1)$. En la práctica, no obstante, no se puede emplear el programa de la figura 1.14 para calcular $n!$ cuando los valores de n son grandes, pues el tamaño de los enteros que se calculen excederá del tamaño de palabra de la máquina en cuestión. $\square$

El método general para resolver una ecuación de recurrencia, tal como se tipifica en el ejemplo 1.10, consiste en reemplazar en forma repetida términos $T(k)$ del lado derecho de la ecuación por el lado derecho completo, donde k se reemplaza por n , hasta obtener una fórmula en la que T no aparezca en el lado derecho como en (1.2). A menudo es necesario calcular la suma de una sucesión o, si no es posible encontrar la suma exacta, obtener una cota superior cercana para la suma a fin de hallar una cota superior para $T(n)$.

Programas con proposiciones GOTO

Hasta ahora, al analizar el tiempo de ejecución de un programa, se supuso de manera tácita que el flujo de control dentro de un procedimiento estaba determinado por construcciones de ciclos y de ramificación. Esto sirvió como base en la determinación del tiempo de ejecución de grupos de proposiciones cada vez más grandes, al suponer que sólo se necesitaba la regla de la suma para agrupar secuencias de proposiciones. Sin embargo, las proposiciones **goto** (proposiciones de transferencia incondicional de control) hacen más complejo el agrupamiento lógico de las proposiciones de un programa. Por esta razón, las proposiciones **goto** deberían evitarse, pero Pascal carece de proposiciones para salir de un ciclo o terminarlo en forma anormal (como *break* y *continue*), por lo que con frecuencia se utiliza **goto** para estos fines.

Para manejar proposiciones **goto** que realicen saltos de un ciclo a código que con seguridad está después del ciclo, se sugiere el siguiente enfoque. (Por lo general,

ésta es la única clase de **goto** que está justificada.) Puesto que es probable que se ejecute el **goto** en forma condicional dentro del ciclo, se puede suponer que nunca se efectúa. Debido a que el **goto** transfiere el control a una proposición que se ejecutará después de terminado el ciclo, esta suposición resulta conservadora; no es posible subestimar el tiempo de ejecución del peor caso si se presume que el ciclo se ejecuta por completo. Sin embargo, es raro el programa en el que ignorar el **goto** es tan conservador que puede llevar a sobreestimar la velocidad de crecimiento del tiempo de ejecución del programa para el peor caso. Obsérvese que si se presentara un **goto** que transfiriera el control a código ejecutado con anterioridad, no sería posible ignorarlo sin correr riesgos, pues ese **goto** podría crear un ciclo que consumiera la mayor parte del tiempo de ejecución.

No por esto debe pensarse que el uso de proposiciones **goto** hacia atrás en sí mismo hace que los tiempos de ejecución sean imposibles de analizar. El enfoque para analizar tiempos de ejecución, que se describió en esta sección, funcionará bien en la medida en que los ciclos de un programa tengan una estructura razonable, es decir, que cualquier par de ciclos sea siempre disjunto o anidado. (No obstante, es responsabilidad del analizador descubrir la estructura de los ciclos.) Así pues, no debe dudarse en aplicar estos métodos de análisis de programas a lenguajes como FORTRAN, donde los **goto** son esenciales, pero los programas tienden a tener una estructura de ciclos razonable.

Análisis de un seudoprograma

Si se conoce la velocidad de crecimiento del tiempo necesario para ejecutar proposiciones informales en español, es posible analizar seudoprogramas como si fueran programas reales. Sin embargo, muchas veces no se conoce el tiempo que demandarán las partes no completamente terminadas de un seudoprograma. Por ejemplo, si se tiene un seudoprograma donde las únicas partes no terminadas son operaciones con TDA, puede elegirse una de las varias implantaciones posibles para un TDA y el tiempo de ejecución total puede depender en gran medida de la realización elegida. Por supuesto, una de las razones en favor de la escritura de programas con TDA es que permite considerar los pros y los contras entre los tiempos de ejecución de varias operaciones que resultan de efectuar distintas realizaciones.

Para analizar seudoprogramas que contengan proposiciones en algún lenguaje de programación y llamadas a procedimientos aún no implantados, tales como operaciones con TDA, se calcula el tiempo de ejecución como una función de los tiempos de ejecución no especificados de esos procedimientos. El tiempo de ejecución de un procedimiento obtiene sus parámetros mediante el «tamaño» de su argumento (o argumentos). Así como sucede con el «tamaño de la entrada», la medida apropiada para el tamaño de un argumento es decisión de quien hace el análisis. Si el procedimiento es una operación con un TDA, el modelo matemático subyacente a menudo indica la noción lógica de tamaño. Por ejemplo, si el TDA está basado en conjuntos, el número de elementos de los conjuntos es con frecuencia la noción correcta de tamaño. En los capítulos siguientes se verán muchos ejemplos de análisis del tiempo de ejecución de seudoprogramas.

1.6 Buenas prácticas de programación

Hay un buen número de ideas que se deberían tener presente al diseñar un algoritmo e implantarlo como programa. Estas ideas a menudo parecen perogrulladas, porque sólo se pueden apreciar en toda su extensión a través de su empleo acertado en situaciones prácticas, más que en el desarrollo de una teoría. A pesar de esto, son lo bastante importantes para repetir las aquí. El lector debe observar con detenimiento la aplicación de estas ideas en los programas diseñados en este libro, así como buscar oportunidades para ponerlas en práctica en sus propios programas.

1. *Planificar el diseño de un programa.* En la sección 1.1 se mostró cómo diseñar un programa partiendo de un esbozo informal del algoritmo, con el diseño de un pseudoprograma y luego con el refinamiento gradual del pseudocódigo hasta convertirlo en ejecutable. Esta estrategia de esbozar, y después detallar, tiende a la producción de un programa final más organizado que facilita su depuración y mantenimiento.
2. *Encapsular.* Empleéanse procedimientos y diversos TDA para que el código asociado a cada operación importante y a cada tipo de datos quede colocado en un solo lugar del listado del programa. Después, si es necesario hacer cambios, la sección de código pertinente se localizará con facilidad.
3. *Usar o modificar un programa existente.* Una de las principales causas de ineficiencia en el proceso de programación es que muchas veces un proyecto se aborda como si fuera el primer programa que se hubiera escrito. Se debería buscar primero un programa ya elaborado que realice la tarea completa o parte de ella. A la inversa, cuando se escribe un programa, se debería pensar en ponerlo a disposición de otras personas para posibles usos no previstos.
4. *Confeccionar herramientas.* En lenguaje de computación, una *herramienta* es un programa que tiene una variedad de usos. Cuando se escriba un programa, debe pensarse en la posibilidad de escribirlo de un modo más general sin mayor esfuerzo adicional. Por ejemplo, supóngase que se pide escribir un programa para la programación de exámenes finales; en lugar de eso, escríbase una herramienta que tome un grafo arbitrario y coloreéense sus vértices con el mínimo de colores posible, de modo que no haya dos vértices del mismo color unidos por una arista. En el contexto de la programación de exámenes, los vértices son los grupos de estudiantes, los colores son los periodos de exámenes, y una arista entre dos grupos significa que éstos tienen algún estudiante en común. El programa de coloración, junto con rutinas que traduzcan las listas de grupos en grafos y los colores en días y horas específicos, es el programador de exámenes. Sin embargo, el programa de coloración se puede usar para resolver problemas que no tengan relación alguna con la programación de exámenes, como el problema de los semáforos de la sección 1.1.
5. *Programar a nivel de mandatos.* En ocasiones, es imposible encontrar en una biblioteca el programa que se necesita para realizar cierto trabajo, ni adaptar una herramienta para tal efecto. Un sistema operativo bien diseñado permite conectar entre sí una red de programas disponibles sin que sea necesario escribir un programa nuevo, sino sólo una lista de mandatos (*commands*) del sistema ope-

rativo. Para que los mandatos sean pertinentes, es necesario que cada uno se comporte como un *filtro*, o sea como un programa con un archivo de entrada y otro de salida. Obsérvese que es posible componer un número arbitrario de filtros, y si el sistema operativo está diseñado de forma inteligente, una simple lista de mandatos (dispuestos como han de ejecutarse) será suficiente como programa.

Ejemplo 1.11. Como ejemplo de lo anterior, considérese el programa *spell*, tal como fue escrito por S. C. Johnson a partir de mandatos de UNIX †. Este programa toma como entrada un archivo a_1 que contiene un texto en inglés, y produce como salida todas aquellas palabras en a_1 que no se encuentren en un pequeño diccionario ††. *spell* tiende a listar los nombres propios como faltas de ortografía, al igual que las palabras que no figuren en el diccionario, pero dado que la salida típica del programa es pequeña, puede examinarse visualmente, y la inteligencia de cualquier ser humano puede determinar si una palabra de la salida de *spell* tiene faltas de ortografía o no. (La ortografía de la versión original en inglés de este libro se verificó con *spell*.)

El primer filtro utilizado por *spell* es un mandato llamado *translate*; cuando este mandato recibe parámetros apropiados, puede reemplazar las letras mayúsculas por minúsculas y los espacios por saltos de línea, sin alterar los caracteres restantes. La salida de *translate* es un archivo a_2 que contiene las palabras de a_1 , sin mayúsculas y una en cada línea. A continuación, aparece el mandato *sort* que clasifica las líneas de su archivo de entrada en orden lexicográfico (alfabético). La salida de *sort* es un archivo a_3 que tiene todas las palabras de a_2 en orden alfabético, con repeticiones. Entonces, un mandato *unique* elimina las líneas duplicadas de su archivo de entrada y produce un archivo de salida a_4 que contiene las palabras del archivo original, sin mayúsculas ni duplicaciones, en orden alfabético. Por último, se aplica a a_4 un mandato *diff*, con un parámetro que indica un segundo archivo a_5 que contiene una lista alfabetizada de las palabras del diccionario, una en cada línea. El resultado tiene todas las palabras de a_4 (y por tanto de a_1) que no están en a_5 , esto es, todas las palabras de la entrada original que no se encuentran en el diccionario. El programa *spell* completo no es más que la siguiente secuencia de mandatos.

```
spell: translate [A-Z] → [a-z], espacio → nueva_línea
      sort
      unique
      diff diccionario
```

La programación a nivel de mandatos requiere disciplina por parte de una comunidad de programadores; éstos deben escribir programas como filtros, y escribir herramientas, en lugar de programas de propósito especial, siempre que les sea posible. La recompensa de esto, en función de la razón global entre trabajo y resultados, es sustancial. □

† UNIX es marca registrada de los Laboratorios Bell.

†† Podría usarse un diccionario no abreviado, pero muchos de los errores ortográficos pueden ser palabras en desuso.

1.7 Súper Pascal

La mayoría de los programas de este libro están escritos en Pascal. No obstante, para aumentar su legibilidad, en ocasiones se usan tres construcciones que no forman parte de Pascal estándar, pero que se pueden traducir mecánicamente a Pascal puro. Una de tales construcciones es la etiqueta no numérica. Las pocas veces que se requieran etiquetas, serán no numéricas, porque facilitan la comprensión de los programas. Por ejemplo, «**goto salida**» siempre es más claro que «**goto 561**». Para convertir a Pascal puro un programa que contenga etiquetas no numéricas, se debe reemplazar cada una de ellas por una etiqueta numérica distinta y después declarar las etiquetas numéricas al principio del programa. Este proceso puede llevarse a cabo mecánicamente.

La segunda construcción no estándar es la proposición de retorno, que se usa porque permite escribir programas más fáciles de entender, sin emplear proposiciones **goto** para interrumpir el flujo de control. La proposición de retorno que aquí se emplea tiene la forma

return (*expresión*)

donde (*expresión*) es opcional. Es fácil convertir a Pascal estándar un procedimiento que contenga proposiciones **return**. En primer lugar, se declara una etiqueta nueva, como 999, y con ella se etiqueta la última proposición **end** del procedimiento. Si en una función de nombre *y*, por ejemplo, aparece la proposición **return**(*x*), ésta se reemplaza por el bloque

```
begin
  y := x;
  goto 999
end
```

En un procedimiento, la proposición **return**, que puede carecer de argumento, simplemente se reemplaza por **goto 999**.

Ejemplo 1.12. La figura 1.15 muestra el programa para calcular factoriales, escrito con proposiciones **return**. En la figura 1.16 aparece el programa en Pascal resultante de la aplicación sistemática de la transformación indicada a la figura 1.15. □

```
function fact ( n: integer ) : integer;
begin
  if n <= 1 then
    return (1)
  else
    return (n * fact(n-1))
  end; {fact}
```

Fig. 1.15. Programa de cálculo de factoriales con proposiciones de retorno.

```

function fact ( n: integer ) : integer;
  label
    999;
  begin
    if n <= 1 then
      begin
        fact := 1;
        goto 999
      end
    else
      begin
        fact := n * fact(n-1);
        goto 999
      end
    999;
  end: { fact }

```

Fig. 1.16. Programa resultante en Pascal.

La tercera extensión a Pascal que se utiliza consiste en usar expresiones como nombres de tipos a lo largo de un programa de manera uniforme. Por ejemplo, una expresión como $\uparrow$ tipo_celda, aunque puede emplearse en cualquier otra parte, no se permite como tipo de un parámetro de un procedimiento o como tipo del valor devuelto por una función. Técnicamente, Pascal requiere que se invente un nombre para la expresión de tipo, como ap_a_celda. En este libro se permitirá este uso de las expresiones de tipo, y se sugiere inventar un nombre para el tipo y que en forma mecánica se reemplacen las expresiones por los nombres. Así, se escribirán proposiciones como

function *y* (*A*: array [1..10] of integer): $\uparrow$ tipo_celda

en lugar de

function *y* (*A*: arreglo_de_diez_enteros): ap_a_celda

Para concluir, debe hacerse una observación acerca de las convenciones tipográficas que se utilizan en los programas. Las palabras reservadas de Pascal se escriben en negritas; los tipos, en redondas, y los nombres de variables, en cursivas. Se hará distinción entre letras mayúsculas y minúsculas.

Ejercicios

- 1.1 Una liga de fútbol tiene seis equipos que se representarán con las letras *A*, *B*, *C*, *D*, *E* y *F*. El equipo *A* ya jugó contra los equipos *B* y *C*; el equipo *B*,

además, se enfrentó al D y al F . El E ha jugado contra el C y el F . Cada equipo tiene un encuentro por semana. Elabórese un calendario tal que cada equipo haya jugado contra todos los demás en el menor número posible de semanas. *Sugerencia:* Diseñese un grafo cuyos vértices representen pares de equipos que aún no se hayan enfrentado. ¿Cuáles deberían ser las aristas para que en una coloración lícita del grafo, cada color pudiera representar los juegos realizados en una semana?

- *1.2 Considérese un brazo robot fijo por un extremo. El brazo tiene dos articulaciones; en cada una de ellas, es posible rotarlo 90 grados hacia arriba o hacia abajo en un plano vertical. ¿Cómo podría modelarse matemáticamente la variedad de movimientos posibles del extremo del brazo? Describese un algoritmo para mover el extremo del brazo robot de una posición permisible a otra.
- *1.3 Supóngase que se desean multiplicar cuatro matrices de números reales: $M_1 \times M_2 \times M_3 \times M_4$, donde M_1 es de 10 por 20, M_2 de 20 por 50, M_3 de 50 por 1 y M_4 de 1 por 100. Supóngase que la multiplicación de una matriz de $p \times q$ por otra de $q \times r$ requiere pqr operaciones escalares, como sucede con el algoritmo usual de multiplicación de matrices. Encuéntrese el orden óptimo de multiplicación de matrices que minimice el número total de operaciones escalares. ¿Cómo se encontraría ese ordenamiento óptimo si se tuviera un número arbitrario de matrices?
- **1.4 Supóngase que se desean dividir las raíces cuadradas de los enteros del 1 al 100 en dos pilas de 50 números cada una, de modo que la suma de los números de la primera pila sea lo más cercana posible a la suma de los números de la segunda. Si sólo se dispusiera de dos minutos de tiempo de computador para resolver el problema, ¿qué cálculo sería aconsejable realizar?
- 1.5 Describese un algoritmo ávido para jugar al ajedrez. ¿Cabe esperar que tenga un buen rendimiento?
- 1.6 En la sección 1.2, se consideró el TDA CONJUNTO con las operaciones ANULA, UNION y TAMAÑO. Supóngase por conveniencia que todos los conjuntos son subconjuntos de $\{0, 1, \dots, 31\}$, y que el TDA CONJUNTO se interpreta como el tipo de datos de Pascal *set of 0..31*. Escribanse procedimientos en PASCAL para esas operaciones utilizando esta realización de CONJUNTO.
- 1.7 El *máximo común divisor* de dos enteros p y q es el mayor entero d que divide exactamente p y q . Se desea desarrollar un programa para calcular el máximo común divisor de dos enteros p y q con el siguiente algoritmo. Sea r el residuo de dividir p entre q . Si r es igual a cero, entonces q es el máximo común divisor. En caso contrario, hágase que p tome el valor de q , y q el de r , y repítase el proceso.
 - a) Muéstrase que este proceso encuentra el máximo común divisor correcto.

- b) Refínese el algoritmo en la forma de un programa enseudolenguaje.
 c) Conviértase el programa enseudolenguaje en un programa en Pascal.

1.8 Se desea desarrollar un programa para formato de textos que coloque las palabras en las líneas de modo que queden justificadas a la izquierda y a la derecha. El programa tiene dos áreas de almacenamiento transitorio (*buffers*), uno de palabras y otro de líneas; ambas están vacías al inicio. Una palabra se lee al área transitoria, y si hay espacio suficiente en el área de líneas, la palabra se transfiere a ésta. En caso contrario, se insertan espacios adicionales entre las palabras del área transitoria de líneas para completar una línea; al imprimir esta línea, el área se vacía.

- a) Refínese este algoritmo en la forma de un programa enseudolenguaje.
 b) Conviértase el programa enseudolenguaje en un programa en Pascal.

1.9 Considérese un conjunto de n ciudades y una tabla de distancias entre pares de ellas. Escríbase un programa enseudolenguaje que encuentre un camino corto que pase por cada ciudad sólo una vez y vuelva a la ciudad en que empezó. No se conoce un método que no realice una búsqueda exhaustiva para obtener el recorrido más corto. Así pues, inténtese encontrar un algoritmo eficiente para este problema utilizando algún procedimiento heurístico razonable.

1.10 Considérense las siguientes funciones de n :

$$f_1(n) = n^2$$

$$f_2(n) = n^2 + 1000n$$

$$f_3(n) = \begin{cases} n & \text{si } n \text{ es impar} \\ n^3 & \text{si } n \text{ es par} \end{cases}$$

$$f_4(n) = \begin{cases} n & \text{si } n \leq 100 \\ n^3 & \text{si } n > 100 \end{cases}$$

Para cada posible par de valores de i y j , indíquese si $f_i(n)$ es $O(f_j(n))$ o no, y si $f_i(n)$ es $\Omega(f_j(n))$ o no.

1.11 Considérense las siguientes funciones de n :

$$g_1(n) = \begin{cases} n^2 & \text{para } n \text{ par} \geq 0 \\ n^3 & \text{para } n \text{ impar} \geq 1 \end{cases}$$

$$g_2(n) = \begin{cases} n & \text{para } 0 \leq n \leq 100 \\ n^3 & \text{para } n > 100 \end{cases}$$

$$g_3(n) = n^{2.5}$$

Para cada par de valores de i y j , indíquese si $g_i(n)$ es $O(g_j(n))$ o no, y si $g_i(n)$ es $\Omega(g_j(n))$ o no.

1.12 Mediante la notación asintótica, obténganse los tiempos de ejecución del peor caso supuesto para cada uno de los procedimientos siguientes como una función de n .

- a) **procedure** *prod_mat* (n : integer);
 var
 i, j, k : integer;
 begin
 for $i := 1$ **to** n **do**
 for $j := 1$ **to** n **do begin**
 $C[i, j] := 0$;
 for $k := 1$ **to** n **do**
 $C[i, j] := C[i, j] + A[i, k] * B[k, j]$
 end
 end
 end
- b) **procedure** *misterio* (n : integer);
 var
 i, j, k : integer;
 begin
 for $i := 1$ **to** $n - 1$ **do**
 for $j := i + 1$ **to** n **do**
 for $k := 1$ **to** j **do**
 { alguna proposición que requiera tiempo $O(1)$ }
 end
 end
 end
- c) **procedure** *muy_impar* (n : integer);
 var
 i, j, x, y : integer;
 begin
 for $i := 1$ **to** n **do**
 if *odd*(i) **then begin**
 for $j := i$ **to** n **do**
 $x := x + 1$;
 for $j := 1$ **to** i **do**
 $y := y + 1$
 end
 end
 end
- *d) **function** *recursiva* (n : integer) : integer;
 begin
 if $n \leq 1$ **then**
 return (1)
 else
 return (*recursiva*($n - 1$) + *recursiva*($n - 1$))
 end

- 1.13** Muéstrese que las siguientes afirmaciones son verdaderas.
- 17 es $O(1)$.
 - $n(n-1)/2$ es $O(n^2)$.
 - $\max(n^3, 10n^2)$ es $O(n^3)$.
 - $\sum_{i=1}^n i^k$ es $O(n^{k+1})$ y $\Omega(n^{k+1})$ para k entero.
 - Si $p(x)$ es cualquier polinomio de grado k donde el coeficiente del término de mayor grado es positivo, entonces $p(n)$ es $O(n^k)$ y $\Omega(n^k)$.
- *1.14** Supóngase que $T_1(n)$ es $\Omega(f(n))$ y $T_2(n)$ es $\Omega(g(n))$. ¿Son verdaderas las siguientes afirmaciones?
- $T_1(n) + T_2(n)$ es $\Omega(\max(f(n), g(n)))$.
 - $T_1(n)T_2(n)$ es $\Omega(f(n)g(n))$.
- *1.15** Algunos autores definen la notación omega mayúscula de la siguiente manera: $f(n)$ es $\Omega(g(n))$ si existen n_0 y $c > 0$ tales que para todo $n \geq n_0$ se tiene que $f(n) \geq cg(n)$.
- ¿Es verdadero para esta definición que $f(n)$ es $\Omega(g(n))$ si, y sólo si, $g(n)$ es $O(f(n))$?
 - ¿Es a) verdadero para la definición de omega mayúscula que se dio en la sección 1.4?
 - ¿Se cumplen con esta definición de omega mayúscula las afirmaciones a) y b) del ejercicio 1.14?
- 1.16** Ordénense las siguientes funciones de acuerdo con su velocidad de crecimiento: a) n , b) $\sqrt{n}$, c) $\log n$, d) $\log \log n$, e) $\log^2 n$, f) $n/\log n$, g) $\sqrt{n} \log^2 n$, h) $(1/3)^n$, i) $(3/2)^n$, j) 17.
- 1.17** Supóngase que el parámetro n del procedimiento siguiente es una potencia positiva de 2, esto es, $n = 2, 4, 8, 16, \dots$ Proporcionese una fórmula que exprese el valor de la variable *cuenta* en función del valor de n cuando termina el procedimiento.

```

procedure misterio (n: integer);
var
  x, cuenta: integer;
begin
  cuenta := 0;
  x := 2;
  while x < n do begin
    x := 2 * x;
    cuenta := cuenta + 1
  end;
  writeln (cuenta)
end

```

- 1.18 La siguiente es una función $máx(i, n)$ que devuelve el mayor elemento en las posiciones i a $i+n-1$ de una matriz entera A . Por conveniencia, puede suponerse que n es una potencia de 2.

```

function  $máx(i, n: integer): integer;$ 
  var
     $m1, m2: integer;$ 
  begin
    if  $n = 1$  then
      return ( $A[i]$ )
    else begin
       $m1 := máx(i, n \div 2);$ 
       $m2 := máx(i+n \div 2, n \div 2);$ 
      if  $m1 < m2$  then
        return ( $m2$ )
      else
        return ( $m1$ )
      end
    end
  end

```

- Sea $T(n)$ el tiempo del peor caso para $máx$ con segundo argumento igual a n . Esto es, n es el número de elementos de entre los cuales se encuentra el mayor. Escribese una ecuación que exprese $T(n)$ en función de $T(j)$ para uno o más valores de j menores que n y una constante o constantes que representen los tiempos tomados por las proposiciones individuales del programa $máx$.
- Obténgase una cota asintótica superior (notación «o mayúscula») lo más cercana posible para $T(n)$. La solución debe ser igual a una cota asintótica inferior (notación «omega mayúscula»), y ser lo más sencilla posible.

Notas bibliográficas

El origen del concepto de tipo de datos abstracto se remonta al tipo *class* en el lenguaje SIMULA 67 (Birtwistle *et al.* [1973]). Desde entonces, se han desarrollado varios lenguajes que manejan tipos de datos abstractos; entre ellos están Alphard (Shaw, Wulf y London [1977]), C con clases (Stroustrup [1982]), CLU (Liskov *et al.* [1977]), MESA (Geschke, Morris y Satterthwaite [1977]) y Russell (Demers y Donahue [1979]). El concepto de TDA se analiza con más profundidad en trabajos tales como Gotlieb y Gotlieb [1978] y Wulf *et al.* [1981].

Knuth [1968] fue el primer trabajo importante en el que se preconizó el estudio sistemático del tiempo de ejecución de programas. Aho, Hopcroft y Ullman [1974] relaciona la complejidad espacial y temporal de los algoritmos con varios modelos computacionales, como las máquinas de Turing y las de acceso aleatorio. Para más referencias sobre el tema del análisis de algoritmos y programas, véanse también las notas bibliográficas del capítulo 9.

Como material adicional sobre programación estructurada véanse Hoare, Dahl y Dijkstra [1972], Wirth [1973], Kernighan y Plauger [1974], y Yourdon y Constantine [1975]. Los problemas organizacionales y psicológicos que se presentan en el desarrollo de proyectos grandes de software se tratan en Brooks [1974] y Weinberg [1971]. En Kernighan y Plauger [1981] se muestra cómo construir herramientas útiles para un ambiente de programación.

2

Tipos de datos abstractos fundamentales

En este capítulo se estudiarán algunos tipos de datos abstractos fundamentales. Se considerarán las listas, que son secuencias de elementos, y dos tipos especiales de listas: las pilas, donde los elementos se insertan y eliminan sólo en un extremo, y las colas, donde los elementos se insertan por un extremo y se eliminan por el otro. Se estudiará después, en forma breve, la correspondencia o memoria asociativa (*mapping*), un TDA que se comporta como una función. Para cada uno de estos TDA, se considerarán varias realizaciones y se analizarán sus méritos relativos.

2.1 El tipo de datos abstracto «lista»

Las listas constituyen una estructura flexible en particular, porque pueden crecer y acortarse según se requiera; los elementos son accesibles y se pueden insertar y suprimir en cualquier posición de la lista. Las listas también pueden concatenarse entre sí o dividirse en sublistas; se presentan de manera rutinaria en aplicaciones como recuperación de información, traducción de lenguajes de programación y simulación. Las técnicas de administración de memoria del tipo de las que se analizan en el capítulo 12 hacen uso extensivo de técnicas de procesamiento de listas. En esta sección se presentarán algunas operaciones básicas con listas, y en el resto del capítulo se presentarán estructuras de datos para representar listas que manejan con eficiencia varios subconjuntos de estas operaciones.

Matemáticamente, una *lista* es una secuencia de cero o más elementos de un tipo determinado (que por lo general se denominará *tipo_elemento*). A menudo se representa una lista como una sucesión de elementos separados por comas

$$a_1, a_2, \dots, a_n$$

donde $n \geq 0$ y cada a_i es del tipo *tipo_elemento*. Al número n de elementos se le llama *longitud* de la lista. Al suponer que $n \geq 1$, se dice que a_1 es el *primer elemento* y a_n el *último elemento*. Si $n = 0$, se tiene una *lista vacía*, es decir, que no tiene elementos.

Una propiedad importante de una lista es que sus elementos pueden estar ordenados en forma lineal de acuerdo con sus posiciones en la misma. Se dice que a_i *precede* a a_{i+1} para $i = 1, 2, \dots, n-1$, y que a_i *sucede* a a_{i-1} para $i = 2, 3, \dots, n$. Se dice que el elemento a_i está en la *posición* i . Es conveniente postular también la existencia de

una posición que sucede a la del último elemento de la lista. La función $\text{FIN}(L)$ devolverá la posición que sigue a la posición n en una lista L de n elementos. Obsérvese que la posición $\text{FIN}(L)$, con respecto al principio de la lista, está a una distancia que varía conforme la lista crece o se reduce, mientras que las demás posiciones guardan una distancia fija con respecto al principio de la lista.

Para formar un tipo de datos abstracto a partir de la noción matemática de lista, se debe definir un conjunto de operaciones con objetos de tipo LISTA $\dagger$. Como sucede con muchos de los TDA que se analizan en este libro, ningún conjunto de operaciones es adecuado para todas las aplicaciones. Aquí se dará un conjunto representativo de operaciones. En la siguiente sección se mostrarán varias estructuras de datos para representar listas y se escribirán procedimientos para las operaciones características con listas en función de esas estructuras de datos.

Para ilustrar algunas operaciones comunes con listas, considérese una aplicación típica: se tiene una lista de direcciones y se desean eliminar las entradas dobles. En forma conceptual, este problema es bastante fácil de resolver: para cada elemento de la lista, elimínense todos los elementos sucesores equivalentes. Para presentar este algoritmo, sin embargo, es necesario definir operaciones que permitan encontrar el primer elemento de una lista, recorrer los demás elementos sucesivos, y recuperar y eliminar elementos.

Se presentará ahora un conjunto representativo de operaciones con listas. Ahí, L es una lista de objetos de tipo `tipo_elemento`, x es un objeto de ese tipo y p es de tipo `posición`. Obsérvese que «posición» es otro tipo de datos cuya implantación cambiará con aquella que se haya elegido para las listas. Aunque de manera informal se piensa en las posiciones como enteros, en la práctica pueden tener otra representación.

1. **INSERTA** (x, p, L). Esta función inserta x en la posición p de la lista L , pasando los elementos de la posición p y siguientes a la posición inmediata posterior. Esto quiere decir que si L es $a_1, a_2, \dots, a_n$, se convierte en $a_1, a_2, \dots, a_{p-1}, x, a_p, \dots, a_n$. Si p es $\text{FIN}(L)$, entonces L se convierte en $a_1, a_2, \dots, a_n, x$. Si la lista L no tiene posición p , el resultado es indefinido.
2. **LOCALIZA** (x, L). Esta función devuelve la posición de x en la lista L . Si x figura más de una vez en L , la posición de la primera aparición de x es la que se devuelve. Si x no figura en la lista, entonces se devuelve $\text{FIN}(L)$.
3. **RECUPERA** (p, L). Esta función devuelve el elemento que está en la posición p de la lista L . El resultado no está definido si $p = \text{FIN}(L)$ o si L no tiene posición p . Obsérvese que si se utiliza **RECUPERA**, los elementos deben ser de un tipo que pueda ser devuelto por una función. No obstante, en la práctica siempre es posible modificar **RECUPERA** para devolver un apuntador a un objeto de tipo `tipo_elemento`.
4. **SUPRIME** (p, L). Esta función elimina el elemento en la posición p de la lista L . Si L es $a_1, a_2, \dots, a_n$, L se convierte en $a_1, a_2, \dots, a_{p-1}, a_{p+1}, \dots, a_n$. El resultado no está definido si L no tiene posición p o si $p = \text{FIN}(L)$.

$\dagger$ En sentido estricto, el tipo es «LISTA de tipo_elemento». Sin embargo, las realizaciones de listas que se proponen no dependen del tipo_elemento; precisamente esta independencia es la que justifica la relevancia que se da al concepto de lista. Se usará «LISTA» en lugar de «LISTA de tipo_elemento», y se tratará de manera similar a los demás TDA que dependen de los tipos de elementos.

5. **SIGUIENTE**(p, L) y **ANTERIOR** (p, L) devuelven las posiciones siguiente y anterior, respectivamente, a p en la lista L . Si p es la última posición de L , **SIGUIENTE**(p, L) = **FIN**(L). **SIGUIENTE** no está definida si p es **FIN**(L). **ANTERIOR** no está definida si p es 1. Ambas funciones están indefinidas cuando L no tiene posición p .
6. **ANULA**(L). Esta función ocasiona que L se convierta en la lista vacía y devuelve la posición **FIN**(L).
7. **PRIMERO**(L). Esta función devuelve la primera posición de la lista L . Si L está vacía, la posición que se devuelve es **FIN**(L).
8. **IMPRIME_LISTA**(L). Imprime los elementos de L en su orden de aparición en la lista.

Ejemplo 2.1. Con estos operadores se escribirá un procedimiento **PURGA** que toma como argumento una lista y elimina sus elementos con doble entrada. Los elementos son de tipo `tipo_elemento`, y una lista de ellos es de tipo **LISTA**; esta convención se seguirá en todo el capítulo. Hay una función *mismo*(x, y), donde x e y son de tipo `tipo_elemento`, que es verdadera si x e y son «el mismo» elemento, y es falsa en caso contrario. El significado de «ser el mismo» es intencionalmente impreciso. Si `tipo_elemento` es real, por ejemplo, podría desearse que *mismo*(x, y) fuera verdadera si, y sólo si, $x = y$. Sin embargo, si `tipo_elemento` es un registro que contiene el número de cuenta, el nombre y la dirección de un suscriptor como en

```

type
  tipo_elemento = record
    núm_cta: integer;
    nombre: packed array [1..20] of char;
    dirección: packed array [1..50] of char
  end

```

por consiguiente podría desearse que *mismo*(x, y) fuera verdadera siempre que $x.\text{núm_cta} = y.\text{núm_cta}$ †.

La figura 2.1 muestra el código de **PURGA**. Las variables p y q se usan para representar dos posiciones en la lista. Conforme el programa avanza, se eliminan de la lista todas las entradas dobles de cualquier elemento a la izquierda de la posición p . En una iteración del ciclo (2)-(8), q se usa para revisar la lista a partir de la posición p , con el objeto de eliminar cualquier entrada doble del elemento en la posición p . Después, p avanza a la siguiente posición y el proceso se repite.

En la siguiente sección, se proporcionarán las declaraciones apropiadas para **LISTA** y posición, y una aplicación de las operaciones, de manera que **PURGA** se convierta en un programa ejecutable. Como ya se dijo, el programa es independiente de la forma en que se representen las listas, por lo que hay libertad para experimentar con varias realizaciones de listas.

† En este caso, si se eliminan los registros que son «el mismo», podría desearse verificar que los nombres y direcciones fueran iguales; si los números de cuenta fueran iguales, pero los otros datos no, es posible que se haya asignado inadvertidamente el mismo número de cuenta a dos personas. No obstante, es más probable que el mismo suscriptor aparezca más de una vez en la lista, con diferentes números y con los demás datos ligeramente distintos. En tales casos, resulta difícil eliminar las entradas dobles.

```
procedure PURGA ( var L: LISTA ) ;  
  {PURGA elimina los elementos duplicados de la lista L}  
  var  
    p, q: posición; { p será la posición "actual" en L, y q  
                     avanzará para encontrar elementos iguales }  
  begin  
    (1)    p := PRIMERO(L);  
    (2)    while p <> FIN(L) do begin  
    (3)      q := SIGUIENTE(p, L);  
    (4)      while q <> FIN(L) do  
    (5)        if mismo(RECUPERA(p, L), RECUPERA(q, L)) then  
    (6)          SUPRIME(q, L)  
              else  
    (7)        q := SIGUIENTE(q, L);  
    (8)      p := SIGUIENTE(p, L)  
    end  
  end;  
end; { PURGA }
```

Fig. 2.1. Programa para eliminar duplicados.

Un aspecto que es importante observar está relacionado con el cuerpo del ciclo interno, las líneas (4)-(7) de la figura 2.1. Cuando se elimina el elemento de la posición q de la línea (6), los elementos que estaban en las posiciones $q+1$, $q+2$, ..., etcétera, avanzan una posición en la lista. En particular, si q fuera la última posición de L , el valor de q sería entonces $\text{FIN}(L)$. Si después de esto se ejecutara la línea (7), $\text{SIGUIENTE}(\text{FIN}(L), L)$ produciría un resultado indefinido. Por tanto, es esencial que sólo alguna de las líneas, (6) o (7), pero no las dos, se ejecute entre las pruebas para saber si $q = \text{FIN}(L)$ en la línea (4). $\square$

2.2 Realización de listas

En esta sección se describirán algunas estructuras de datos que pueden utilizarse para representar listas. Se considerarán realizaciones de listas basadas en arreglos, apuntadores y cursores. Cada una de ellas permite realizar ciertas operaciones con listas de manera más eficiente que otras.

Realización de listas mediante arreglos

En la realización de una lista mediante arreglos, los elementos de ésta se almacenan en celdas contiguas de un arreglo. Esta representación permite recorrer con facilidad una lista y agregarle elementos nuevos al final. Pero insertar un elemento en la mitad de la lista obliga a desplazarse una posición dentro del arreglo a todos los elementos que siguen al nuevo elemento para concederle espacio. De la misma for-

ma, la eliminación de un elemento, excepto el último, requiere desplazamientos de elementos para llenar de nuevo el vacío formado.

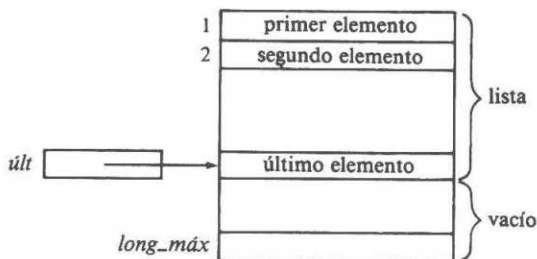


Fig. 2.2. Realización de una lista mediante un arreglo.

En la realización con arreglos se define el tipo LISTA como un registro con dos campos; el primero es un arreglo de elementos que tiene la longitud adecuada para contener la lista de mayor tamaño que se pueda presentar. El segundo campo es un entero *últ* que indica la posición del último elemento de la lista en el arreglo. El *i*-ésimo elemento de la lista está en la *i*-ésima celda del arreglo, para $1 \leq i \leq \text{últ}$, como se muestra en la figura 2.2. Las posiciones en la lista se representan mediante enteros; la *i*-ésima posición, mediante el entero *i*. La función FIN(*L*) sólo tiene que devolver *últ* + 1. Las declaraciones importantes son:

```

const
  long_máx = 100 { alguna constante apropiada };
type
  LISTA = record
    elementos: array[1..long_máx] of tipo_elemento;
    últ: integer
  end;
  posición = integer;
function FIN (var L: LISTA): posición†;
begin
  return (L.últ + 1)
end; {FIN }

```

La figura 2.3 muestra cómo se podrían implantar las operaciones INSERTA, SUPRIME y LOCALIZA con esta realización basada en arreglos. INSERTA pasa los elementos de las localidades *p*, *p* + 1, ..., *últ* a las localidades *p* + 1, *p* + 2, ..., *últ* + 1 y después inserta el nuevo elemento en la localidad *p*. Si no hay espacio en el arreglo para insertar un nuevo elemento, se invoca a la rutina *error*, que imprime su argumen-

† Aunque *L* no se modifica, pasa por referencia, pues por lo común es una estructura grande y no se desea perder tiempo copiándola.

```

procedure INSERTA ( x: tipo_elemento; p: posición; var L: LISTA );
{ INSERTA coloca x en la posición p de la lista L }
var
    q: posición;
begin
    if L.últ > = long_máx then
        error ('la lista está llena')
    else if (p > L.últ + 1) or (p < 1) then
        error ('la posición no existe')
    else begin
        for q := L.últ downto p do
            { desplaza los elementos en p, p + 1,... una posición hacia abajo }
            L.elementos [q + 1] := L.elementos[q];
        L.últ := L.últ + 1;
        L.elementos[p] := x
    end
end; { INSERTA }

procedure SUPRIME ( p: posición; var L: LISTA );
{ SUPRIME elimina el elemento en la posición p de la lista L }
var
    q: posición;
begin
    if (p > L.últ) or (p < 1) then
        error ('la posición no existe')
    else begin
        L.últ := L.últ - 1;
        for q := p to L.últ do
            { desplaza los elementos en p + 1, p + 2... una posición hacia arriba }
            L.elementos[q] := L.elementos[q + 1]
        end
    end; { SUPRIME }

function LOCALIZA ( x: tipo_elemento; var L: LISTA ) : posición;
{ LOCALIZA devuelve la posición de x en la lista L }
var
    q: posición;
begin
    for q := 1 to L.últ do
        if L.elementos[q] = x then
            return (q);
    return (L.últ + 1) { si no se encuentra }
end; { LOCALIZA }

```

• Fig. 2.3. Realización basada en arreglos de algunas operaciones con listas.

to y da por terminada la ejecución del programa. SUPRIME elimina el elemento de la posición p pasando los elementos de las posiciones $p + 1, p + 2, \dots, \text{últ}$, a las posiciones $p, p + 1, \dots, \text{últ} - 1$. LOCALIZA revisa el arreglo secuencialmente, en busca de un elemento determinado; si no lo encuentra, LOCALIZA devuelve $\text{últ} + 1$.

Debería estar claro cómo codificar las restantes operaciones con listas con esta forma de implantación. Por ejemplo, PRIMERO siempre devuelve uno; SIGUIENTE devuelve uno más que su argumento, y ANTERIOR, uno menos; cada uno verifica primero si el resultado está dentro del intervalo permisible; ANULA(L) coloca $L.\text{últ}$ en 0. Si el procedimiento PURGA de la figura 2.1 va precedido de

1. las definiciones de tipo_elemento y de la función mismo.
2. las definiciones de LISTA, posición y FIN(L) anteriores,
3. la definición de SUPRIME de la figura 2.3, y
4. definiciones adecuadas para los procedimientos triviales PRIMERO, SIGUIENTE y RECUPERA,

entonces se obtiene un procedimiento PURGA ejecutable.

Al principio puede parecer tedioso escribir procedimientos que rijan todos los accesos a las estructuras subyacentes de un programa. Sin embargo, si se logra establecer la disciplina de escribir programas en función de operaciones de manipulación de tipos de datos abstractos en lugar de usar ciertos detalles de implantación particulares, es posible modificar los programas más fácilmente con sólo aplicar de nuevo las operaciones, en lugar de buscar en todos los programas aquellos lugares donde se hacen accesos a las estructuras de datos subyacentes. La flexibilidad que se obtiene con esto, puede ser especialmente importante en proyectos grandes, y no se debe poner en tela de juicio este concepto por los ejemplos, necesariamente pequeños, que se encuentran en este libro.

Realización de listas mediante apuntadores

La segunda forma de realización de listas, celdas enlazadas sencillas, utiliza apuntadores para enlazar elementos consecutivos. Esta implantación permite eludir el empleo de memoria contigua para almacenar una lista y, por tanto, también elude los desplazamientos de elementos para hacer inserciones o rellenar vacíos creados por la eliminación de elementos. No obstante, por esto hay que pagar el precio de un espacio adicional para los apuntadores.

En esta representación, una lista está formada por celdas; cada celda contiene un elemento de la lista y un apuntador a la siguiente celda. Si la lista es $a_1, a_2, \dots, a_n$, la celda que contiene a_i tiene un apuntador a la celda que contiene a a_{i+1} , para $i = 1, 2, \dots, n - 1$. La celda que contiene a_n posee un apuntador nil. Existe también una celda de encabezamiento que apunta a la celda que contiene a_1 ; esta celda de encabezamiento no tiene ningún elemento†. En el caso de una lista vacía, el apuntador del encabe-

† El empleo de una celda completa para el encabezamiento simplifica la implantación de operaciones con listas en Pascal. Es posible usar apuntadores como encabezamientos si se pretende aplicar operaciones de modo que las inserciones y las eliminaciones al principio de la lista se manejen de manera especial. Véase el análisis de este método en esta misma sección, en el apartado que explica la realización de listas mediante cursores.

zamiento es *nil* y no se tienen más celdas. La figura 2.4 muestra una lista enlazada de esta manera.

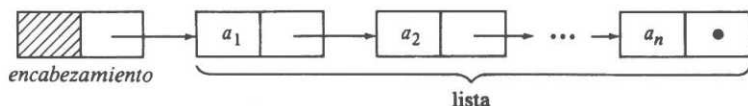


Fig. 2.4. Lista enlazada.

Para las listas enlazadas sencillas es conveniente usar una definición de posición ligeramente distinta de la que se empleó para las listas logradas mediante arreglos. Aquí, la posición i será un apuntador a la celda que contiene el apuntador a a_i para $i = 2, 3, \dots, n$. La posición 1 es un apuntador al encabezamiento, y la posición $\text{FIN}(L)$ es un apuntador a la última celda de L .

El tipo de una lista, en realidad, es el mismo que el de una posición; es un apuntador a una celda particular: el encabezamiento. La siguiente es una definición formal de las partes esenciales de una estructura de datos de lista enlazada.

```

type
    tipo_celda = record
        elemento: tipo_elemento;
        sig: ↑ tipo_celda
    end;
    LISTA = ↑ tipo_celda;
    posición = ↑ tipo_celda;
    
```

En la figura 2.5 se muestra la función $\text{FIN}(L)$; ésta funciona moviendo el apuntador q por la lista, hasta que alcanza el final, el cual se detecta por el hecho de que q apunta a una celda con un apuntador *nil*. Obsérvese que esta realización de FIN es ineficiente, pues requiere revisar la lista entera cada vez que se desea calcular $\text{FIN}(L)$. Si es necesario hacer esto con frecuencia, como en el programa PURGA de la figura 2.1, puede optarse por cualquiera de las opciones siguientes:

```

function FIN ( L: LISTA ) : posición;
{ FIN devuelve un apuntador a la última celda de L }
var
    q: posición;
begin
    (1)  q := L;
    (2)  while q↑.sig <> nil do
    (3)      q := q↑.sig;
    (4)  return (q)
end; { FIN }
    
```

Fig. 2.5. La función FIN .

1. usar una representación de listas que incluya un apuntador de la última celda, o
2. sustituir el uso de $\text{FIN}(L)$ donde sea posible. Por ejemplo, la condición $p \neq \text{FIN}(L)$ de la línea (2) de la figura 2.1 podría reemplazarse por $p \uparrow.\text{sig} \neq \text{nil}$, al costo de que ese programa dependa de una realización particular de listas.

La figura 2.6 contiene rutinas para las operaciones INSERTA, SUPRIME, LOCALIZA y ANULA usando esta realización de listas con apuntadores. Las operaciones restantes se pueden obtener como rutinas de un solo paso, con la excepción de ANTERIOR, que requiere revisar la lista desde el principio. Estas rutinas se dejan como ejercicio para el lector. Obsérvese que muchos de los mandatos no necesitan el parámetro L , la lista, por lo que se omite.

En la figura 2.7 se muestra la mecánica del manejo de apuntadores en el procedimiento INSERTA de la figura 2.6. La figura 2.7 (a) muestra la situación antes de la ejecución de INSERTA. Se desea insertar un nuevo elemento ante la celda que contiene b , por lo que p es un apuntador a la celda de la lista que contiene el apuntador a b . En la línea (1), temp apunta a la celda que contiene b . En la línea (2), se crea una nueva celda de la lista y se hace que el campo sig de la celda que contiene a apunte hacia esta misma celda. En la línea (3) se almacena x en el campo elemento de la celda recién creada, y en la línea (4), el campo sig toma el valor de temp y, de esta manera, apunta a la celda que contiene b . La figura 2.7 (b) muestra el resultado de la ejecución de INSERTA. Los apuntadores nuevos se representan con líneas punteadas y se marca el paso en el cual fueron creados.

El procedimiento SUPRIME es más sencillo. La figura 2.8 muestra las manipulaciones de apuntadores que realiza el procedimiento SUPRIME de la figura 2.6. Los apuntadores antiguos se representan por medio de líneas de trazo continuo, y los nuevos por medio de líneas punteadas.

Obsérvese que una posición en una implantación de listas enlazadas se comporta de distinta forma que una posición en una realización de arreglos. Supóngase que se tiene una lista con tres elementos a, b, c y una variable p de tipo posición que tiene actualmente el valor 3; es decir, apunta a la celda que contiene b y, por tanto, representa la posición de c . Si se ejecuta un mandato para insertar x en la posición 2, la lista se convierte en a, x, b, c , con lo cual el elemento b pasaría a ocupar la posición 3. Si se utilizara la realización de arreglos descrita anteriormente, b y c se moverían hacia el principio del arreglo, de modo que b ocuparía en realidad la tercera posición.

procedure INSERTA (x : tipo_elemento; p : posición);

var

temp: posición;

begin

- (1) temp := $p \uparrow.\text{sig}$;
 - (2) new($p \uparrow.\text{sig}$);
 - (3) $p \uparrow.\text{sig}.\text{elemento}$:= x ;
 - (4) $p \uparrow.\text{sig}.\text{sig}$:= temp
- end;** { INSERTA }

```

procedure SUPRIME ( p: posición );
begin
    p↑.sig := p↑.sig↑.sig
end; { SUPRIME }

function LOCALIZA ( x: tipo_elemento; L: LISTA ) : posición;
var
    p: posición;
begin
    p := L;
    while p↑.sig <> nil do
        if p↑.sig↑.elemento = x then
            return (p)
        else
            p := p↑.sig;
    return (p) { si no es encontrado }
end; { LOCALIZA }

function ANULA ( var L: LISTA ) : posición;
begin
    new (L);
    L↑.next := nil;
    return(L)
end; { ANULA }

```

Fig. 2.6. Algunas operaciones con la realización de listas enlazadas.

Pero, si se usara la realización de listas enlazadas, el valor de *p*, que es un apunador a la celda que contiene *b*, no cambiaría como consecuencia de la inserción, de modo que después de ésta el valor de *p* sería la «posición 4», no la 3. Esta variable de posición debe actualizarse si se va a usar después como posición de *b* †.

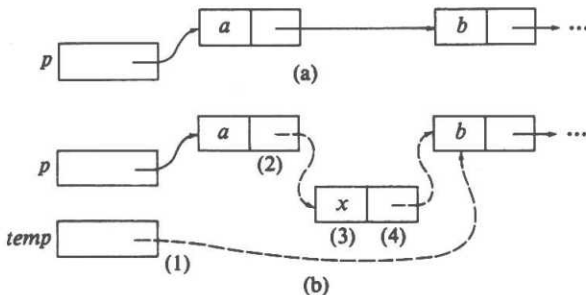


Fig. 2.7. Diagrama de INSERTA.

† Por supuesto, existen muchas situaciones en las que se podría desear que *p* represente la posición de *c*.

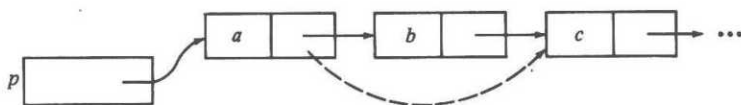


Fig. 2.8. Diagrama de SUPRIME.

Comparación de los métodos

Cabe preguntarse si en determinadas circunstancias es mejor usar una realización de listas basada en apuntadores o una basada en arreglos. A menudo, la respuesta depende de cuáles son las operaciones que se desea realizar, o las que se realizan más frecuentemente. Otras veces, la decisión depende de lo larga que puede llegar a ser la lista. Los aspectos más importantes a tener en cuenta son los siguientes.

1. La realización con arreglos requiere especificar el tamaño máximo de una lista en el momento de la compilación. Si no es posible acotar la probable longitud de la lista, quizá deba optarse por una realización con apuntadores.
2. Ciertas operaciones son más lentas en una realización que en otra. Por ejemplo, INSERTA y SUPRIME tienen un número constante de pasos para una lista enlazada, pero cuando se utiliza la realización con arreglos, requieren un tiempo proporcional al número de elementos que siguen. A la inversa, la ejecución de ANTERIOR y FIN requiere un tiempo constante con la realización con arreglos, pero se usa un tiempo proporcional a la longitud de la lista si se usan apuntadores.
3. Si un programa efectúa inserciones o supresiones que afecten al elemento que ocupa la posición denotada por alguna variable de posición, y el valor de esa variable se va a utilizar posteriormente, entonces la representación con apuntadores no se puede usar de la forma descrita aquí. Como principio general, los apuntadores se deben utilizar con gran cuidado y moderación.
4. La realización con arreglos puede malgastar espacio, puesto que usa la cantidad máxima de espacio, independientemente del número de elementos que en realidad tiene la lista en un momento dado. La realización con apuntadores utiliza sólo el espacio necesario para los elementos que actualmente tiene la lista, pero requiere espacio para el apuntador en cada celda. Así, cualquiera de los dos métodos podría usar más espacio que el otro, dependiendo de las circunstancias.

Realización de listas basadas en cursores

Algunos lenguajes, como FORTRAN y ALGOL, no tienen apuntadores. Si se trabaja con un lenguaje tal, se pueden simular los apuntadores mediante cursores; esto es, con enteros que indican posiciones en arreglos. Se crea un arreglo de registros para

almacenar todas las listas de elementos cuyo tipo es tipo_elemento; cada registro contiene un elemento y un entero que se usa como cursor. Es decir, se define

```
var
    ESPACIO: array [1..long_máx] of record
        elemento: tipo_elemento;
        sig: integer
    end
```

Si L es una lista de elementos, se declara una variable entera, por ejemplo, $Lencab$, como encabezamiento para L . Se puede tratar $Lencab$ como un cursor a la celda de encabezamiento en $ESPACIO$ con un campo *elemento* vacío. Las operaciones con listas se pueden implantar, entonces, como en la realización basada en apuntadores recién descrita.

Aquí se describirá una realización alternativa que evita el uso de celdas de encabezamiento, al tomar como casos especiales las inserciones y supresiones en la posición 1. Esta técnica se puede usar también con listas enlazadas basadas en apuntadores, para evitar el empleo de celdas de encabezamiento. Para una lista L , el valor de $ESPACIO[Lencab].elemento$ es el valor del primer elemento de L . El valor de $ESPACIO[Lencab].sig$ es el índice de la celda que contiene el segundo elemento, y así sucesivamente. Un valor de 0 ya sea en $Lencab$ o en el campo *sig*, representa un «apuntador nil», esto es, no hay un elemento siguiente.

Una lista tendrá tipo entero, ya que el encabezamiento es una variable entera que representa la lista como un todo. Las posiciones serán también de tipo entero. Se adopta aquí la convención de que la posición i de la lista L es el índice de la celda de $ESPACIO$ que contiene el elemento $i - 1$ de L , ya que el campo *sig* de esa celda contendrá el cursor al elemento i . Como caso especial, la posición 1 de cualquier lista se representa por 0. Dado que el nombre de la lista es siempre un parámetro de las operaciones que usan posiciones, es posible distinguir entre las primeras posiciones de distintas listas. La posición $FIN(L)$ es el índice del último elemento de L .

La figura 2.9 muestra dos listas, $L = a, b, c$ y $M = d, e$, que comparten el arreglo $ESPACIO$ con $long_máx = 10$. Obsérvese que las celdas del arreglo no contenidas

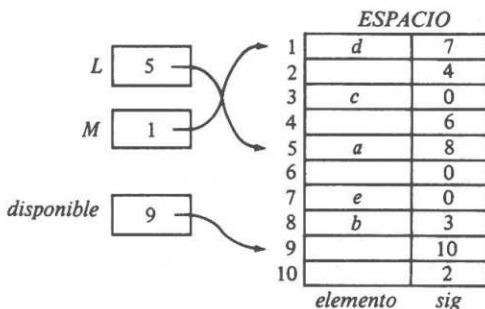


Fig. 2.9. Realización de listas enlazadas mediante cursores.

en L ni en M están enlazadas en otra lista llamada *disponible*. Tal lista es necesaria para obtener una celda vacía cuando se desee hacer una inserción, y disponer de lugar donde poner las celdas suprimidas para su uso posterior.

Para insertar un elemento x en la lista L , se toma la primera celda de la lista *disponible* y se coloca en la posición correcta en la lista L . El elemento x se pone entonces en el campo *elemento* de esta celda. Para suprimir un elemento x de la lista L , se quita de L la celda que contiene x y se devuelve al principio de la lista *disponible*. Estas dos acciones pueden verse como casos especiales del acto de tomar una celda C apuntada por un cursor p y hacer que otro cursor q apunte a C , a la vez que se hace que p quede apuntando hacia donde C apuntaba y C quede apuntando hacia donde q apuntaba. De esta forma, C se inserta entre q y aquello hacia lo que apuntaba q . Por ejemplo, si se suprime b de la lista L en la figura 2.9, C es la fila 8 de *ESPACIO*, p es *ESPACIO*[5].sig y q es *disponible*. Los cursores antes (en líneas de trazo continuo) y después (en líneas punteadas) de esta acción se muestran en la figura 2.10, y el código está incluido en la función *mueve* de la figura 2.11, que realiza el movimiento si C existe y devuelve falso si C no existe.

La figura 2.12 muestra los procedimientos *INSERTA* y *SUPRIME*, y un procedimiento *val_inicial* que enlaza las celdas del arreglo *ESPACIO* para formar una lista de espacio disponible. Estos procedimientos no incluyen verificaciones para detectar errores; es aconsejable insertarlas como ejercicio para el lector. Se dejan también como ejercicios otras operaciones similares a las de las listas enlazadas basadas en apuntadores.

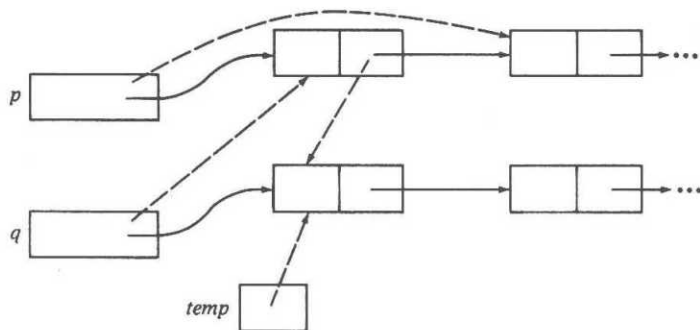


Fig. 2.10. Cambio de una celda C de una lista a otra.

```
function mueve ( var p, q: integer ) : boolean;
{ mueve coloca la celda apuntada por p delante de la celda apuntada por q }
var
    temp: integer;
begin
```

```

if  $p = 0$  then begin { celda inexistente }
    writeln('la celda no existe');
    return (false)
end
else begin
    temp := q;
    q := p;
    p := ESPACIO[q].sig;
    ESPACIO[q].sig := temp;
    return (true)
end
end; { mueve }

```

Fig. 2.11. Código para cambiar una celda.

Listas doblemente enlazadas

En algunas aplicaciones puede ser deseable recorrer eficientemente una lista, tanto hacia adelante como hacia atrás. O, dado un elemento, podría desearse determinar con rapidez el siguiente y el anterior. En tales situaciones, quizá se quisiera poner en cada celda de una lista un apuntador a la siguiente celda y otro a la anterior, como se sugiere en la lista doblemente enlazada de la figura 2.13. En el capítulo 12 se mencionan algunas situaciones específicas en que las listas doblemente enlazadas son esenciales por razones de eficiencia.

```

procedure INSERTA ( x: tipo_elemento; p: posición; var L: LISTA );
begin
    if  $p = 0$  then begin
        { inserta en la primera posición }
        if mueve ( disponible, L ) then
            ESPACIO [L]. elemento := x
    end
    else { inserta en una posición que no es la primera }
        if mueve ( disponible, ESPACIO [p].sig ) then
            { celda que ocupará x ya apuntada por ESPACIO [p].sig }
            ESPACIO [ESPACIO [p].sig]. elemento := x
    end; { INSERTA }

procedure SUPRIME ( p: posición; var L: LISTA );
begin
    if  $p = 0$  then
        mueve ( L, disponible )
    else
        mueve ( ESPACIO [p].sig, disponible )
    end; { SUPRIME }

```

```

procedure val_inicial;
{ val_inicial enlaza ESPACIO en una lista de celdas disponibles }
var
    i: integer;
begin
    for i := long_máx-1 downto 1 do
        ESPACIO [i].sig := i + 1;
    disponible := 1;
    ESPACIO [long_máx].sig := 0 { marca el final de la lista de celdas disponibles }
end; { val_inicial }

```

Fig. 2.12. Algunos procedimientos para listas enlazadas basadas en cursores.

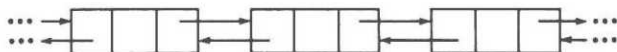


Fig. 2.13. Una lista doblemente enlazada.

Otra ventaja importante de las listas doblemente enlazadas es que permiten usar un apuntador a la celda que contiene el i -ésimo elemento de una lista para representar la posición i , en vez de usar el apuntador a la celda anterior, que es menos natural. El único precio que se paga por estas características es la presencia de un apuntador adicional en cada celda, y los procedimientos algo más lentos para algunas de las operaciones básicas con listas. Si se usan apuntadores (en vez de cursores) se puede declarar que las celdas contienen un elemento y dos apuntadores, mediante

```

type
    tipo_celda = record
        elemento: tipo_elemento;
        sig, ant: ↑ tipo_celda
    end;
    posición = ↑ tipo_celda;

```

En la figura 2.14 se da un procedimiento para suprimir un elemento en la posición p en una lista doblemente enlazada. La figura 2.15 muestra los cambios causados en los apuntadores por este procedimiento; los apuntadores anteriores se representan con líneas de trazo continuo, y los nuevos con líneas punteadas, en el supuesto de que la celda suprimida no es la primera ni la última †. Primero se localiza la celda precedente, usando el campo *ant*. Se hace que el campo *sig* de esta celda apunte a

† A tal efecto, es práctica común hacer que el encabezamiento de una lista doblemente enlazada sea una celda que efectivamente «cierre el círculo». Esto es, que el campo *ant* del encabezado apunte a la última celda y que su campo *sig* apunte a la primera. De esta manera no se necesita hacer verificaciones de campos nil en la figura 2.14.

la celda que sigue a la que ocupa la posición p . Después se hace que el campo *ant* de esta celda siguiente apunte a la celda que precede a la que ocupa la posición p . La celda apuntada por p queda sin usar y el sistema de tiempo de ejecución de Pascal debe de usarla de nuevo automáticamente.

```

procedure SUPRIME (var  $p$ : posición );
begin
  if  $p↑.ant \neq nil$  then
    { la celda a suprimir no es la primera }
     $p↑.ant↑.sig := p↑.sig$ ;
  if  $p↑.sig \neq nil$  then
    { la celda a suprimir no es la última }
     $p↑.sig↑.ant := p↑.ant$ 
end; { SUPRIME }

```

Fig. 2.14. Supresión en una lista doblemente enlazada.

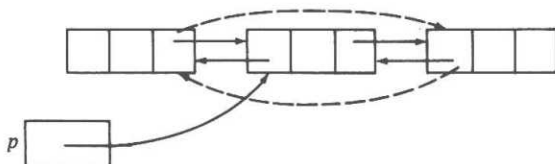


Fig. 2.15. Cambios de los apuntadores para realizar una supresión.

2.3 Pilas

Una *pila* es un tipo especial de lista en la que todas las inserciones y supresiones tienen lugar en un extremo denominado *tope*. A las pilas se les llama también «listas LIFO» (*last in first out*) o listas «último en entrar, primero en salir». El modelo intuitivo de una pila es precisamente una pila de fichas de póquer puesta sobre una mesa, o de libros sobre el piso, o de platos en una estantería, situaciones todas en las que sólo es conveniente quitar o agregar un objeto del extremo superior de la pila, al que se denominará en lo sucesivo «tope». Un tipo de datos abstracto de la familia PILA incluye a menudo las cinco operaciones siguientes:

1. **ANULA**(P) convierte la pila P en una pila vacía. Esta operación es exactamente la misma que para las listas generales.
2. **TOPE**(P) devuelve el valor del elemento de la parte superior de la pila P . Si se identifica la parte superior de una pila con la posición 1, como suele hacerse, entonces **TOPE**(P) puede escribirse en función de operaciones con listas como **RECUPERA**(**PRIMERO**(P), P).
3. **SACA**(P), en inglés *POP*, suprime el elemento superior de la pila, es decir, equivale a **SUPRIME**(**PRIMERO**(P), P). Algunas veces resulta conveniente implantar

SACA como una función que devuelve el elemento que acaba de suprimir, aunque aquí no se hará eso.

4. METE(x , P), en inglés *PUSH*, inserta el elemento x en la parte superior de la pila P . El anterior tope se convierte en el siguiente elemento, y así sucesivamente. En función de operaciones primitivas con listas, esta operación es INSERTA(x , PRIMERO(P), P).
5. VACIA(P) devuelve verdadero si la pila P está vacía, y falso en caso contrario.

Ejemplo 2.2. Los editores de textos siempre permiten usar un carácter (por ejemplo, *back-space*) como *carácter de borrado* que cancela el carácter anterior no cancelado. Por ejemplo, si '#' es el carácter de borrado, la cadena *abc#d##e* es en realidad la cadena *ae*. El primer '#' cancela la *c*, el segundo la *d* y el tercero la *b*.

Los editores de textos también tienen un *carácter de eliminación de línea*, que cancela todos los caracteres anteriores de la línea actual. A efectos de este ejemplo, se usará '@' como carácter de eliminación de línea.

Un editor puede procesar una línea de texto usando una pila. El editor lee un carácter a la vez. Si el carácter leído no es de borrado ni de eliminación de línea, el editor lo mete en la pila. Si el carácter es de borrado, el editor saca un carácter de la pila, y si es de eliminación de línea, vacía la pila. En la figura 2.16 se muestra un programa que ejecuta estas acciones.

```

procedure EDITA;
  var
    P: PILA;
    c: char;
  begin
    ANULA (P);
    while not eoln do begin
      read(c);
      if c = '#' then
        SACA (P)
      else if c = '@' then
        ANULA (P)
      else { c es un carácter normal }
        METE (c, P)
    end;
    imprime P en orden inverso
  end; { EDITA }

```

Fig. 2.16. Programa que logra el efecto de los caracteres de borrado y cancelación de línea.

En este programa, el tipo PILA debe declararse como una lista de caracteres. El proceso de escribir la pila en orden inverso en la última línea del programa tiene un objetivo: sacar de la pila un carácter a la vez da como resultado la inversión de la

línea. Algunas realizaciones de pilas, como la basada en arreglos, que se analizará a continuación, permiten escribir un procedimiento sencillo para imprimir los caracteres de la pila a partir de la base. Sin embargo, en general, para invertir una pila, cada elemento debe sacarse y meterse en otra pila; luego, los elementos pueden sacarse de la segunda pila e imprimirse en el orden en que se sacan. □

Realización de pilas basada en arreglos

Todas las realizaciones de listas descritas sirven para pilas, puesto que una pila con sus operaciones es un caso especial de lista con sus operaciones. La representación de una pila como lista enlazada es sencilla, pues METE y SACA operan sólo con la celda de encabezamiento y con la primera celda de la lista. De hecho, los encabezamientos pueden ser apuntadores o cursores, más que celdas completas, puesto que para las pilas no existe la noción de «posición» para las listas y, por tanto, no es necesario representar la posición 1 en forma análoga a las demás posiciones.

Sin embargo, la realización de listas basada en arreglos que se dio en la sección 2.3 no es particularmente adecuada para las pilas, dado que cada METE o SACA tiene que mover la lista entera hacia arriba o hacia abajo, lo cual lleva un tiempo proporcional al número de elementos de la pila. Un mejor criterio para usar un arreglo es el que tiene en cuenta el hecho de que las inserciones y las supresiones ocurren sólo en la parte superior. Se puede anclar la base de la pila a la base del arreglo (el extremo de índice más alto) y dejar que la pila crezca hacia la parte superior del arreglo (el extremo de índice más bajo). Un cursor llamado *tope* indicará la posición actual del primer elemento de la pila. Esta idea se ilustra en la figura 2.17.

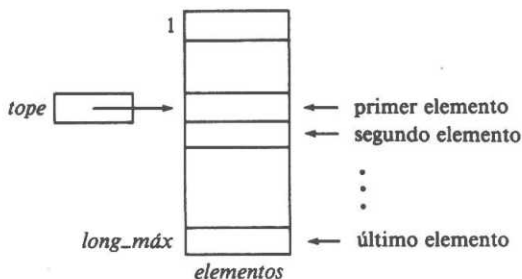


Fig. 2.17. Realización de una pila mediante un arreglo.

Para esta realización de pilas basada en arreglos, el tipo de datos abstracto PILA se define como

```
type
  PILA = record
    tope: integer;
    elementos: array[1..long_máx] of tipo_elemento
  end;
```

Un ejemplo de la pila consiste en la sucesión $elementos[tope]$, $elementos[tope + 1]$, ..., $elementos[long_máx]$. Obsérvese que si $tope = long_máx + 1$, la pila está vacía.

Las cinco operaciones típicas con pilas están realizadas en la figura 2.18. Obsérvese que para que TOPE devuelva un objeto de tipo *tipo_elemento*, ese tipo debe ser lícito como resultado de una función. De lo contrario, TOPE debe ser un procedimiento que modifique su segundo argumento asignándole el valor $TOPE(P)$ o una función que devuelva un apuntador a *tipo_elemento*.

```

procedure ANULA ( var P: PILA);
  begin
     $P.tope := long\_máx + 1$ 
  end; { ANULA }

function VACIA ( P: PILA ): boolean;
  begin
    if  $P.tope > long\_máx$  then
      return (true)
    else
      return (false)
    end; { VACIA }

function TOPE ( var P: PILA ): tipo_elemento;
  begin
    if VACIA (P) then
      error ('la pila está vacía')
    else
      return ( $P.elementos[P.tope]$ )
    end; { TOPE }

procedure SACA ( var P: PILA );
  begin
    if VACIA(P) then
      error ('la pila está vacía')
    else
       $P.tope := P.tope + 1$ 
    end; { SACA }

procedure METE ( x: tipo_elemento; P: PILA);
  begin
    if  $P.tope := 1$  then
      error ('la pila está llena')
    else begin
       $P.tope := P.tope - 1$ ;
       $P.elementos[P.tope] := x$ 
    end
  end; { METE }

```

Fig. 2.18. Operaciones con pilas.

2.4 Colas

Una *cola* es otro tipo especial de lista en el cual los elementos se insertan en un extremo (el *posterior*) y se suprimen en el otro (el *anterior* o *frente*). Las colas se conocen también como listas «FIFO» (*first-in first-out*) o listas «primero en entrar, primero en salir». Las operaciones para una cola son análogas a las de las pilas; las diferencias sustanciales consisten en que las inserciones se hacen al final de la lista, y no al principio, y en que la terminología tradicional para colas y listas no es la misma. Se usarán las siguientes operaciones con colas.

1. ANULA(*C*) convierte la cola *C* en una lista vacía.
2. FRENTE(*C*) es una función que devuelve el valor del primer elemento de la cola *C*. FRENTE(*C*) se puede escribir en función de operaciones con listas, como RECUPERA(PRIMERO(*C*), *C*).
3. PONE_EN_COLA(*x*, *C*) inserta el elemento *x* al final de la cola *C*. En función de operaciones con listas, PONE_EN_COLA(*x*, *C*) es INSERTA(*x*, FIN(*C*), *C*).
4. QUITA_DE_COLA(*C*) suprime el primer elemento de *C*; es decir, QUITA_DE_COLA(*C*) es SUPRIME(PRIMERO(*C*), *C*).
5. VACIA(*C*) devuelve verdadero si, y sólo si, *C* es una cola vacía.

Realización de colas basada en apuntadores

Igual que en el caso de las pilas, cualquier realización de listas es lícita para las colas. No obstante, para aumentar la eficacia de PONE_EN_COLA es posible aprovechar el hecho de que las inserciones se efectúan sólo en el extremo posterior. En lugar de recorrer la lista de principio a fin cada vez que se desea hacer una inserción, se puede mantener un apuntador (o un cursor) al último elemento. Como en las listas de cualquier clase, también se mantiene un apuntador al frente de la lista; en las colas, ese apuntador es útil para ejecutar mandatos del tipo FRENTE o QUITA_DE_COLA. En Pascal, se utilizará una celda ficticia como encabezamiento y se tendrá el apuntador frontal dirigido a ella. Esta convención permite manejar convenientemente una cola vacía.

Aquí se desarrollará una implantación de colas basada en apuntadores de Pascal. Se puede desarrollar una realización análoga basada en cursores, pero en el caso de las colas se dispone de una representación orientada a arreglos, mejor que la que se puede obtener por imitación directa de apuntadores mediante cursores. Al final de esta sección se analizará otra implantación, llamada de «arreglos circulares». Para la realización basada en apuntadores, se definirán las celdas como antes:

```
type
  tipo_celda = record
    elemento: tipo_elemento;
    sig: ↑ tipo_celda
  end;
```

Por tanto, es posible definir una cola como una estructura que consiste en apuntadores al extremo anterior de la lista y al extremo posterior. La primera celda de una cola es una celda de encabezamiento cuyo campo elemento se ignora. Como se mencionó, esta convención permite representar de manera simple una cola vacía. Se define:

```
type
  COLA = record
    ant, post: ↑ tipo_celda
  end;
```

La figura 2.19 muestra programas para las cinco operaciones con colas. En ANULA, la primera proposición *new(C.ant)* crea una variable de tipo *tipo_celda* y asigna su dirección a *C.ant*. La segunda proposición coloca *nil* en el campo *sig* de esa celda. La tercera proposición hace que el encabezamiento quede como primera y última celda de la cola.

El procedimiento QUITA-DE-COLA(*C*) suprime el primer elemento de *C* desconectando el encabezado antiguo de la cola. El primer elemento de la lista se convierte en la nueva celda ficticia de encabezamiento.

La figura 2.20 muestra los resultados originados por la sucesión de mandatos ANULA(*C*), PONE-EN-COLA(*x*, *C*), PONE-EN-COLA(*y*, *C*), QUITA-DE-COLA(*C*). Obsérvese que después de la supresión, el elemento *x* ya no se considera parte de la cola, por estar en el campo elemento de la celda de encabezamiento.

Realización de colas con arreglos circulares

La representación de listas por medio de arreglos analizada en la sección 2.2 puede usarse para las colas, pero no es muy eficiente. Es cierto que con un apuntador al último elemento es posible ejecutar PONE-EN-COLA en un número fijo de pasos, pero QUITA-DE-COLA, que suprime el primer elemento, requiere que la cola completa ascienda una posición en el arreglo. Así pues, QUITA-DE-COLA lleva un tiempo $\Omega(n)$ si la cola tiene longitud *n*.

Para evitar este gasto, se debe adoptar un punto de vista diferente. Imagínese un arreglo como un círculo en el que la primera posición sigue a la última, en la forma sugerida en la figura 2.21. La cola se encuentra en alguna parte de ese círculo, ocupando posiciones consecutivas[†], con el extremo posterior en algún lugar a la izquierda del extremo anterior. Para insertar un elemento en la cola, se mueve el apuntador *C.post* una posición en el sentido de las manecillas del reloj, y se escribe el elemento en esa posición. Para suprimir, simplemente se mueve *C.ant* una posición en el sentido de las manecillas del reloj. De esta manera, la cola se mueve en ese mismo sentido conforme se insertan y suprimen elementos. Obsérvese que utilizando

[†] Obsérvese que la palabra «consecutivas» debe interpretarse en un sentido circular. Esto es, una cola de longitud cuatro puede ocupar, por ejemplo, las dos últimas y las dos primeras posiciones del arreglo.